A project report on

PRECISION AGRICULTURE MONITORING QUADRUPED ROBOT DOG

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Robotics)

by

ADITHYA SANKAR MENON (23BRS1079)
SIDDHESH PUSHKRAJ SHINDE (23BCE5048)
RACHIT GUPTA (23BCE1658)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

PRECISION AGRICULTURE MONITORING QUADRUPED ROBOT DOG

Submitted in partial fulfillment for the award of the degree of

Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Robotics)

by

ADITHYA SANKAR MENON (23BRS1079)
SIDDHESH PUSHKRAJ SHINDE (23BCE5048)
RACHIT GUPTA (23BCE1658)



VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

October, 2026

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

DECLARATION

I hereby declare that the thesis entitled "Precision Agriculture Monitoring Quadruped Robot Dog" submitted by Adithya Sankar Menon (23BRS1079), for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Robotics), Vellore Institute of Technology, Chennai is a record of bonafide work carried out by me under the supervision of Dr. Lakshmanan K.

I further declare that the work reported in this thesis has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Chennai

Date: **Signature of the Candidate**

VIT®
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)
CHENNAI

School of Computer Science and Engineering

CERTIFICATE

This is to certify that the report entitled "Precision Agriculture Monitoring Quadruped Robot Dog" is prepared and submitted by Adithya Sankar Menon (23BRS1079) to Vellore Institute of Technology, Chennai, in partial fulfillment of the requirement for the award of the degree of **Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Robotics)** is a bonafide record carried out under my guidance. The project fulfills the requirements as per the regulations of this University and in my opinion meets the necessary standards for submission. The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma and the same is certified.

Signature of the Guide:

Name: Dr. Lakshmanan K

Date:

Signature of the Examiner	Signature of the Examiner
Name:	Name:
Date:	Date:

Approved by the Head of Department,

B.Tech. Computer Science and Engineering (Artificial Intelligence and Robotics)

Name: Dr. D. Sathian

Date:

(Seal of the School of Computer Science and Engineering)

ABSTRACT

Monitoring crops at the level of individual plants still depends largely on people walking the field, because drones see the canopy only from above and wheeled field robots struggle in soft soil and narrow furrows. Legged robots can walk between planted rows, but research quadrupeds are built around custom actuators and cost far more than a farm can justify. This project set out to build a low-cost quadruped for in-field crop and air-quality monitoring, using a 3D-printed chassis, hobby servos and commodity electronics.

The platform is a twelve-degree-of-freedom quadruped printed from an adapted open design, with its geometry measured from the printed parts. Sensing is carried by two Wi-Fi microcontroller nodes. A Seeed XIAO ESP32-S3 node reads volatile organic compounds and estimated carbon dioxide (SGP30), temperature (MCP9808), particle count (PPD42) and ultrasonic range, displays readings on an onboard screen and sounds alerts. An ESP32-CAM node streams video. A Flask dashboard on a laptop logs every reading, shows live charts and video, and classifies leaf disease with a MobileNetV2 network trained on the PlantVillage dataset. A colour-based vegetation check runs before the network, so that frames without a leaf are reported as such instead of being given a confident disease label.

Locomotion was developed in simulation. Closed-form inverse kinematics were verified against the physics engine's forward kinematics to below 10^{-7} m. A MuJoCo model built from the measured geometry, with servo torque capped at the MG996R stall value, showed that the robot's centre of mass lies 12 mm behind the body centre; until footholds were placed around the measured centre of mass, the trot toppled in every one of 24 gait settings tried. With that correction the simulated robot surveyed three crop rows autonomously, 16.7 m in 129 s, with body tilt below 2.7°, detected every row end with its front ultrasonic sensor and located a simulated gas source to within 0.2 m. A speed sweep showed that MG996R torque limits the trot to 0.18 m/s.

On the bench, the gas, dust, temperature, video and classification modules worked end to end over Wi-Fi, with video at 16–20 frames per second. On the robot itself the servos move the legs and hold a stance, but cannot yet step.

ACKNOWLEDGEMENT

It is my pleasure to express with deep sense of gratitude to Dr. Lakshmanan K, [Designation], School of Computer Science and Engineering, Vellore Institute of Technology, Chennai, for constant guidance, continual encouragement and understanding throughout this project; more than all, for teaching me patience in my endeavour. My association with my guide is not confined to academics only; it has been a great opportunity on my part to work with an intellectual and expert in the field of robotics and intelligent systems.

It is with gratitude that I would like to extend my thanks to the visionary leader Dr. G. Viswanathan our Honorable Chancellor, Mr. Sankar Viswanathan, Dr. Sekar Viswanathan, Dr. G V Selvam Vice Presidents, Dr. Sandhya Pentareddy, Executive Director, Ms. Kadhambari S. Viswanathan, Assistant Vice-President, Dr. V. S. Kanchana Bhaaskaran Vice-Chancellor, Dr. T. Thyagarajan Pro-Vice Chancellor, VIT Chennai and Dr. P. K. Manoharan, Additional Registrar for providing an exceptional working environment and inspiring all of us during the tenure of the course.

Special mention to Dr. Viswanathan V, Dean, Dr. Nithyanandam P, Associate Dean, Dr. Suganya G, Associate Dean, Dr. Sweetlin Hemalatha C, Associate Dean, School of Computer Science and Engineering, Vellore Institute of Technology, Chennai for spending their valuable time and efforts in sharing their knowledge and for helping us in every aspect.

In jubilant state, I express ingeniously my whole-hearted thanks to Dr. D. Sathian, Head of the Department, B.Tech. Computer Science and Engineering (Artificial Intelligence and Robotics) and the Project Coordinators for their valuable support and encouragement to take up and complete the thesis.

I also thank my teammates, Siddhesh Pushkraj Shinde and Rachit Gupta, for their work on this project. My sincere thanks to all the faculties and staffs at Vellore Institute of Technology, Chennai who helped me acquire the requisite knowledge. I would like to thank my parents for their support. It is indeed a pleasure to thank my friends who encouraged me to take up and complete this task.

Place: Chennai

Date: **Adithya Sankar Menon**

CONTENTS

Chapter 1 Introduction....1
1.1 BACKGROUND....1
1.2 MOTIVATION....2
1.3 PROBLEM STATEMENT....2
1.4 OBJECTIVES....3
1.5 SCOPE OF THE PROJECT....3
1.6 CONTRIBUTIONS....4
1.7 ORGANISATION OF THE REPORT....4
Chapter 2 Literature Survey....5
2.1 INTRODUCTION....5
2.2 LEGGED ROBOTS....5
2.2.1 RESEARCH QUADRUPEDS....5
2.2.2 LOW-COST AND OPEN QUADRUPEDS....5
2.3 ROBOTICS IN AGRICULTURE....6
2.4 SENSING AND IOT IN AGRICULTURE....6
2.4.1 SENSOR NETWORKS....6
2.4.2 VOCS AS DISEASE INDICATORS....7
2.5 IMAGE-BASED PLANT DISEASE DETECTION....7
2.5.1 DEEP LEARNING ON PLANTVILLAGE....7
2.5.2 LIGHTWEIGHT NETWORKS....8
2.5.3 THE CLOSED-SET PROBLEM....8
2.6 PHYSICS SIMULATION FOR LEGGED ROBOTS....8
2.7 SUMMARY OF THE LITERATURE....9
2.8 RESEARCH GAPS....10
Chapter 3 System Requirements and Design....11
3.1 INTRODUCTION....11
3.2 REQUIREMENTS....11
3.2.1 FUNCTIONAL REQUIREMENTS....11
3.2.2 NON-FUNCTIONAL REQUIREMENTS....11
3.3 HARDWARE COMPONENTS....12
3.4 SOFTWARE COMPONENTS....12
3.5 INITIAL ARCHITECTURE....13
3.6 DESIGN REVISION AND RATIONALE....14
3.7 FINAL SYSTEM ARCHITECTURE....15
3.8 COMMUNICATION DESIGN....15

3.9 SUMMARY 16
Chapter 4 Mechanical Design and Fabrication 17
4.1 BASE PLATFORM 17
4.2 FABRICATION 17
4.3 MEASURED GEOMETRY 18
4.4 TEAM-DESIGNED PARTS 19
4.5 STATIC TORQUE ANALYSIS 19
4.6 ASSEMBLED ROBOT 20
4.7 SUMMARY 21
Chapter 5 Embedded Sensing System 22
5.1 INTRODUCTION 22
5.2 SENSOR NODE HARDWARE 22
5.3 ELECTRICAL INTERFACING 23
5.3.1 VOLTAGE DIVIDERS 23
5.3.2 I²C BUS AND POWER 23
5.3.3 WIRING 24
5.4 SENSOR NODE FIRMWARE 24
5.4.1 STRUCTURE AND SCHEDULING 24
5.4.2 GAS SENSING 25
5.4.3 DUST SENSING 25
5.4.4 TEMPERATURE MEASUREMENT 26
5.4.5 ULTRASONIC RANGING 26
5.4.6 DISPLAY, TELEMETRY AND WI-FI 27
5.5 CAMERA NODE 27
5.5.1 HARDWARE 27
5.5.2 FIRMWARE 28
5.5.3 SELF-REGISTRATION 28
5.6 DEVELOPMENT WORKFLOW 28
5.7 SUMMARY 28
Chapter 6 Dashboard and Leaf-Disease Classification 30
6.1 INTRODUCTION 30
6.2 DASHBOARD ARCHITECTURE 30
6.3 USER INTERFACE 31
6.4 VIDEO PIPELINE 31
6.5 CLASSIFICATION MODEL 32
6.6 VEGETATION GATE 32
6.7 IMAGE TAGGING AND LOGGING 33

6.8 TESTING WITHOUT HARDWARE 34
6.9 SUMMARY 34
Chapter 7 Kinematics, Gait and Simulation 35
7.1 INTRODUCTION 35
7.2 LEG KINEMATICS 35
7.2.1 COORDINATE FRAMES 35
7.2.2 CLOSED-FORM INVERSE KINEMATICS 36
7.2.3 VERIFICATION 37
7.3 EVOLUTION OF THE SIMULATION 37
7.3.1 QUASI-STATIC BROWSER SIMULATOR 37
7.3.2 GAZEBO AND ROS 2 37
7.3.3 MUJOCO 38
7.4 MUJOCO MODEL 38
7.5 TROT GAIT 39
7.6 CENTRE-OF-MASS-CENTRED FOOTHOLDS 40
7.7 AUTONOMOUS SURVEY MISSION 41
7.7.1 SIMULATED FIELD 41
7.7.2 ENVIRONMENT AND SENSOR MODELS 41
7.7.3 MISSION LOGIC 42
7.7.4 INTERACTIVE VIEWER AND RECORDING 42
7.8 SUMMARY 43
Chapter 8 Results and Discussion 44
8.1 INTRODUCTION 44
8.2 BENCH VALIDATION OF THE HARDWARE 44
8.3 LOGGED SENSOR SESSIONS 45
8.4 VIDEO STREAMING 46
8.5 LEAF-DISEASE CLASSIFICATION 47
8.6 ACTUATION TRIALS ON THE PHYSICAL ROBOT 50
8.7 KINEMATIC VERIFICATION 52
8.8 GAIT STABILITY 53
8.9 SPEED LIMIT IMPOSED BY THE SERVOS 53
8.10 AUTONOMOUS FIELD SURVEY 54
8.11 COMPARISON WITH THE OBJECTIVES 57
8.12 LIMITATIONS 58
8.13 DISCUSSION 58
8.14 SUMMARY 59
Chapter 9 Conclusion and Future Work 60

9.1 CONCLUSION....60
9.2 FUTURE WORK....61
REFERENCES....62
Appendix 1 Wiring and Pin Map....65
Appendix 2 Telemetry Format and Dashboard Interface....67
Appendix 3 Key Source Code Listings....68
Appendix 4 Operating Instructions....71
Appendix 5 Bill of Materials....72

LIST OF FIGURES

Figure 3.1: System architecture proposed in the early project reviews.....13

Figure 3.2: Final system architecture: two sensing nodes on the chassis and a laptop base station on a shared Wi-Fi hotspot.....15

Figure 4.1: The print-ready part set: body parts (gold), leg parts (grey) and the team-designed mounting parts (blue).....18

Figure 4.2: The assembled robot: sensor node and antenna on the top cover, camera node in the head, payload powered from a USB power bank.....21

Figure 5.1: Sensor node on the XIAO expansion board, showing live gas readings and its network address on the OLED.....22

Figure 5.2: Wiring schematic of the sensor node; every 5 V sensor output reaches the XIAO through a divider.....24

Figure 5.3: Task schedule of the sensor node over one 30-second dust window..25

Figure 6.1: Software architecture of the dashboard.....30

Figure 6.2: Dashboard on 18 October 2026, with live gas, dust and temperature readings from the physical sensor node.....31

Figure 6.3: The vegetation gate on four camera frames from 18 October 2026: leaf-coloured pixels are highlighted in green, and frames below 12 % leaf coverage are rejected before classification.....33

Figure 7.1: Leg geometry in the sagittal plane for a foot 30 mm ahead of the hip at a body height of 180 mm.....36

Figure 7.2: Path of the front-left foot over one gait cycle at 0.14 m/s (left) and its height against gait phase (right).....39

Figure 7.3: Top view during a trot stance phase: the diagonal support line passes through the body centre, in front of the centre of mass.....40

Figure 7.4: Start of the survey: the robot standing at the head of the first row gap, with the live sensor panels and the onboard camera view.....42

Figure 7.5: Turning in the headland between two passes; the trail behind the robot records the TVOC measured along its route.....43

Figure 8.1: Temperature and Wi-Fi signal strength logged from the sensor node over 28 minutes on 12 October 2026.....45

Figure 8.2: TVOC, eCO_2 (logarithmic scale) and temperature logged from the sensor node over 17 minutes on 14 October 2026; dashed lines mark the moderate and high TVOC thresholds.....46

Figure 8.3: Scan of a hedge from about a metre away: leaf coverage 11 %, just below the 12 % threshold, so the classifier's 75.5 % grape black rot is withheld and the user is asked to bring a leaf closer.....48

Figure 8.4: Scan of a potted plant with long strap-shaped leaves: leaf coverage 60 %, labelled healthy corn (maize) at 53.4 %, just above the 0.5 reporting threshold.....49

Figure 8.5: Scan of a photograph of spotted leaves displayed on a screen: leaf coverage 26 %, labelled grape black rot at 80.5 %.....49

Figure 8.6: Leg actuation test with the robot on its back, frames from a 10 s video: the legs move through their commanded joint angles with no load on the feet 51

Figure 8.7: Standing test, frames from a 10 s video: the robot supports its own weight and shifts its legs, but slowly and with little reserve 52

Figure 8.8: Achieved speed (line) and share of time a servo was at its torque limit (bars) against commanded speed; the cross marks a fall.....54

Figure 8.9: Simulated robot crossing the stressed patch; the TVOC reading is in the moderate band and the trail behind it is red.....55

Figure 8.10: Survey route coloured by measured TVOC over the modelled VOC field; squares mark row ends detected by the front ultrasonic sensor 56

Figure 8.11: Simulated sensor readings during the survey; vertical lines mark row ends 56

Figure 8.12: Servo load, body tilt and speed during the survey 57

Figure A1.1: Breadboard layout: power rails fed from the expansion board, and dividers A, B and C for the PPD42 and the two ultrasonic echo signals 65

LIST OF TABLES

Table 2.1: Summary of surveyed work and its relevance to this project 9

Table 3.1: Hardware used in the final system 12

Table 3.2: Software and tools 12

Table 3.3: Changes between the Review III plan and the final system 14

Table 3.4: Message flows between the components 16

Table 4.1: Geometry measured from the printed parts 18

Table 4.2: Static knee torque against the MG996R rating (Review III mass, 1.757 kg) 20

Table 5.1: Sensor node pin allocation 23

Table 5.2: Firmware task periods 25

Table 7.1: Parameters of the MuJoCo model 38

Table 8.1: Bench tests of the physical sensing system 44

Table 8.2: Scans logged by the dashboard 47

Table 8.3: Gait experiments in simulation (2 Hz trot, duty factor 0.6 unless stated) 53

Table 8.4: Trot speed sweep with servo torque capped at 1.0 N·m 53

Table 8.5: Results of the simulated autonomous survey 54

Table 8.6: Objectives and their status 57

Table A1.1: Wire-by-wire connection list 65

Table A2.1: Dashboard HTTP interface 67

Table A2.2: Camera node HTTP interface 67

Table A5.1: Bill of materials 72

LIST OF ACRONYMS

ADC	Analogue-to-Digital Converter
API	Application Programming Interface
CNN	Convolutional Neural Network
CoM	Centre of Mass
CSV	Comma-Separated Values
DOF	Degrees of Freedom
eCO_2	Estimated Carbon Dioxide Equivalent
FK	Forward Kinematics
FPS	Frames Per Second
GPIO	General-Purpose Input/Output
HSV	Hue, Saturation, Value
HTTP	Hypertext Transfer Protocol
I^2C	Inter-Integrated Circuit
IK	Inverse Kinematics
IMU	Inertial Measurement Unit
IoT	Internet of Things
ISR	Interrupt Service Routine
JSON	JavaScript Object Notation
LPO	Low-Pulse Occupancy
MJPEG	Motion JPEG
MJCF	MuJoCo XML Model Format
OLED	Organic Light-Emitting Diode (display)
PLA	Polylactic Acid
PSRAM	Pseudo-Static Random-Access Memory
PWM	Pulse-Width Modulation
ROS 2	Robot Operating System 2
RSSI	Received Signal Strength Indicator
SGP30	Sensirion multi-pixel gas sensor (TVOC / eCO_2)
TPU	Thermoplastic Polyurethane
TVOC	Total Volatile Organic Compounds
URDF	Unified Robot Description Format
VOC	Volatile Organic Compound

Chapter 1
Introduction

1.1 BACKGROUND

Agriculture feeds a growing population from a limited area of arable land, and a substantial fraction of what is planted never reaches harvest. A global expert survey of five staple crops attributed yield losses of 10–28 % in wheat, 25–41 % in rice and 20–41 % in maize to pathogens and pests, with the highest losses concentrated in food-deficit regions (Savary et al., 2019). Much of this damage is avoidable if outbreaks are noticed early, while they are still confined to a few plants. Early notice, however, requires someone or something to look at the crop closely, often and across the whole field.

Precision agriculture addresses this by measuring field conditions directly and acting only where the measurements say action is needed. Over the last decade it has drawn on three technology streams: sensor networks that report microclimate continuously (Ray, 2017; Tzounis et al., 2017), machine learning that turns images and sensor data into decisions (Liakos et al., 2018; Kamilaris & Prenafeta-Boldú, 2018), and field robots that carry instruments to the crop (Bechar & Vigneault, 2016; Fountas et al., 2020). Each stream has a gap at the scale of individual plants. Fixed sensor nodes measure only where they are installed. Drones see the top of the canopy but not the underside of leaves or the air within the rows. Ground robots can get close, but most are wheeled or tracked, and lose traction in wet soil, ruts and narrow furrows.

Legged robots are a natural answer to the terrain problem, because they choose where each foot lands and can step over obstacles that stop a wheel. Research quadrupeds such as MIT Cheetah 3 and ANYmal have shown remarkable mobility (Bledt et al., 2018; Hutter et al., 2016), but they are built around custom high-torque actuators and are priced far beyond a farm budget. At the other end of the scale, open hobby designs printed on a desktop 3D printer and driven by inexpensive servos have become widely available. This project asks whether such a low-cost quadruped can serve as a practical platform for in-field monitoring, and what has to be done to make it work.

1.2 MOTIVATION

Two observations motivated the project. The first is that many indicators of crop stress are local. Diseased and stressed plants change the volatile organic compounds they release (Martinelli et al., 2015), leaf lesions are visible only at close range, and dust from field operations settles unevenly. A sensor that moves along the rows at plant height can capture these signals in a way that a fixed station or a high-flying drone cannot.

The second is that the hardware needed to try this idea has become very cheap. A complete printed quadruped chassis costs a few thousand rupees in filament, Wi-Fi microcontrollers with camera interfaces cost a few hundred rupees, and pretrained image classifiers for plant disease are openly available. What is missing is an integrated, tested system that shows how these parts fit together, where they fall short, and which design decisions matter. Building that system, and measuring it honestly, is the purpose of this work.

1.3 PROBLEM STATEMENT

Current crop monitoring relies on manual scouting, fixed sensor nodes or aerial imagery, none of which gives frequent, close-range, multi-parameter measurements along the planted rows. Commercial legged robots that could carry sensors through those rows are too expensive for most farms, while low-cost printed quadrupeds have not been shown to carry a useful sensing payload or to walk reliably with hobby-grade actuators. Image-based disease classifiers achieve high accuracy on curated leaf photographs but give confident and misleading answers when a field camera shows something other than a leaf.

The problem addressed in this project is therefore to design, build and evaluate a low-cost quadruped monitoring platform that (a) measures air quality, temperature and distance at crop level, (b) captures and classifies leaf images without reporting diseases in frames that contain no leaf, (c) presents all measurements live and keeps a permanent record, and (d) has a locomotion design whose feasibility with inexpensive servos is established quantitatively rather than assumed.

1.4 OBJECTIVES

The project had the following objectives:

1. To fabricate a twelve-degree-of-freedom quadruped chassis by 3D printing, adapt it to carry the sensing payload, and measure its true geometry.
2. To build a wireless sensing payload that measures total volatile organic compounds, estimated carbon dioxide, particle count, temperature and ultrasonic range, with firmware that is robust to wiring and network faults.
3. To stream images from a camera on the robot and classify leaf diseases with a deep learning model, rejecting frames that do not contain a leaf.
4. To develop a live dashboard that displays every measurement, logs it permanently and tags each classified image with the air conditions at the moment of capture.
5. To derive and verify the leg kinematics, and to establish through physics simulation with realistic servo limits whether the robot can walk and survey crop rows autonomously.
6. To evaluate each module quantitatively, and to state clearly what has and has not been demonstrated.

1.5 SCOPE OF THE PROJECT

The project covers the mechanical platform, the embedded sensing and camera nodes, the laptop-side dashboard and classifier, the locomotion design and its simulation. Within this scope, the sensing payload was built and tested on physical hardware, and locomotion was developed and evaluated in simulation. The following are outside the present scope: field trials on a working farm, calibration of the gas and dust sensors against reference instruments, training a new disease classifier on field images, GPS-based localisation, and walking on the physical robot. Basic actuation of the physical legs was tested at the end of the project and is reported, but walking is planned as the next phase.

The scope changed during the project. The plan presented at Review III placed an NVIDIA Jetson Orin Nano on the robot for onboard inference. Analysis of the servo torque margin, the electrical design and the risk of damage to an expensive board in a fall led to a revised design in which two low-cost microcontroller nodes carry the

sensors and inference runs on a laptop. The reasons for this change are discussed in Section 3.6.

1.6 CONTRIBUTIONS

The main contributions of this work are:

- A working, low-cost, two-node wireless sensing payload, with firmware that detects absent sensors, diagnoses network faults and lets the camera register itself with the dashboard without any manual configuration.
- A disease classification pipeline in which a colour-based vegetation check prevents the classifier from labelling non-leaf frames, demonstrated on frames that the classifier alone labelled as diseased with 82–90 % confidence.
- A physics model of the printed quadruped built from measured geometry with servo torque capped at the MG996R stall value, and an autonomous row-survey controller driven by the robot's own range sensor.
- Two design findings that apply to the physical robot: footholds must be placed around the measured centre of mass, which sits 12 mm behind the body centre, and MG996R-class servos limit the trot to about 0.18 m/s.

1.7 ORGANISATION OF THE REPORT

Chapter 2 surveys the literature on legged robots, agricultural robotics, agricultural sensing, image-based disease detection and robot simulation, and identifies the gaps this project addresses. Chapter 3 sets out the requirements, the hardware and software used, the original architecture and the revised one. Chapter 4 describes the mechanical design, fabrication and static torque analysis of the chassis. Chapter 5 covers the embedded sensing system: the sensor node, its interfacing and firmware, and the camera node. Chapter 6 describes the dashboard and the leaf-disease classification pipeline. Chapter 7 presents the leg kinematics, the evolution of the simulation work and the MuJoCo model, gait and survey mission. Chapter 8 reports and discusses the results, and Chapter 9 concludes and outlines future work. Appendices give the wiring map, the telemetry format and dashboard interface, key source listings, operating instructions and the bill of materials.

Chapter 2
Literature Survey

2.1 INTRODUCTION

This chapter reviews the work on which the project builds. It is organised by the five areas the system draws on: legged robots, robotics in agriculture, sensing and the Internet of Things in agriculture, image-based plant disease detection, and physics simulation for legged robots. The chapter closes with a summary of the surveyed work and the research gaps that shaped the project's objectives.

2.2 LEGGED ROBOTS

2.2.1 RESEARCH QUADRUPEDS

Modern quadrupeds reached their current level of agility through advances in actuation and control. Bledt et al. (2018) presented MIT Cheetah 3, which uses proprioceptive actuators, motors with low gear ratios that sense contact forces through motor current, together with model-predictive control to walk and climb stairs without vision. The design emphasises robustness to unexpected contact, which matters for any robot moving through cluttered ground. Hutter et al. (2016) introduced ANYmal, a torque-controllable quadruped of about 30 kg built for robust operation and resistant to impulsive loads during running and jumping. ANYmal has since become a reference platform for field robotics research.

Learning-based control has extended what these robots can do. Hwangbo et al. (2019) trained neural network policies in simulation and transferred them to ANYmal, producing locomotion that followed velocity commands more precisely and efficiently than earlier controllers and could recover from falls. The approach depends on a simulator accurate enough that a policy learned in it still works on the real robot, a point that recurs in Section 2.6.

2.2.2 LOW-COST AND OPEN QUADRUPEDS

A parallel line of work aims to make legged robots accessible. Katz et al. (2019) described Mini Cheetah, a small and comparatively inexpensive quadruped intended for rapid control development. Kau et al. (2019) presented Stanford Doggo, an open-source quasi-direct-drive quadruped that can be built with hand tools for under

US$3,000 and that matched or exceeded the vertical jumping agility of larger robots. Both still rely on brushless motors and custom transmissions.

Below this cost level sit hobby designs printed on desktop 3D printers and driven by standard radio-control servos. The SpotMicro design used in this project belongs to that class (KDY0523, n.d.). Such robots are cheap and easy to reproduce, but their servos have limited torque, no torque sensing and significant backlash, and published designs rarely carry a useful payload or come with a quantitative analysis of whether the servos are strong enough. This project treats that analysis as a central question.

2.3 ROBOTICS IN AGRICULTURE

Bechar & Vigneault (2016) reviewed the concepts and components of agricultural robots for field operations, setting out the sensing, decision-making and actuation that such a robot needs and the reasons field conditions make each one harder than in a factory. Fountas et al. (2020) surveyed agricultural robotics for field operations and found that research effort concentrates on harvesting and weeding. Monitoring, which needs mobility and sensing more than manipulation, has received less attention.

Most of the platforms in these reviews are wheeled or tracked. Wheels are efficient on firm ground but sink and slip in wet soil and need clear, continuous paths between rows. A legged platform trades efficiency for the ability to place each foot, which suits slow monitoring tasks in soft or uneven ground. No surveyed work combined a low-cost legged platform with an environmental and imaging payload for crop monitoring.

2.4 SENSING AND IOT IN AGRICULTURE

2.4.1 SENSOR NETWORKS

Ray (2017) reviewed Internet of Things technologies for smart agriculture, covering sensing, communication and application layers and the practical challenges of deploying them in fields. Elijah et al. (2018) examined how IoT and data analytics are changing agriculture, identifying the shift from standalone wireless sensor networks towards connected systems that combine sensing with analysis and decision support. Tzounis et al. (2017) surveyed recent advances and open challenges, including power supply, connectivity in rural areas and the cost of deploying many nodes.

These systems share a limitation that motivates this project: they monitor fixed points. Covering a field densely requires many nodes, each needing power and maintenance, while a single mobile platform can carry one set of sensors to many locations. The cost of mobility is that each location is measured only when the robot passes it.

2.4.2 VOCS AS DISEASE INDICATORS

Plants release volatile organic compounds continuously, and infection or stress changes both the amount and the mixture. Martinelli et al. (2015) reviewed advanced methods of plant disease detection and included VOC profiling among the techniques that can reveal infection, in some cases before visible symptoms appear. Laboratory methods such as gas chromatography identify individual compounds, but are slow and expensive. Low-cost metal-oxide gas sensors such as the SGP30 cannot distinguish compounds; they report a total VOC concentration and an estimated CO_2 equivalent (Sensirion AG, n.d.). Such a sensor can indicate that something unusual is present at a location, which is the role it plays in this project, but it cannot identify a disease on its own.

2.5 IMAGE-BASED PLANT DISEASE DETECTION

2.5.1 DEEP LEARNING ON PLANTVILLAGE

Hughes & Salathé (2015) released PlantVillage, an open collection of tens of thousands of images of healthy and diseased leaves photographed under controlled conditions. It quickly became the standard benchmark. Mohanty et al. (2016) trained deep convolutional networks on 54,306 PlantVillage images covering 14 crops and 26 diseases, and reported 99.35 % accuracy on held-out images. They also observed that accuracy fell substantially on images taken under conditions unlike the training set, an early warning about deploying such models in the field. Ferentinos (2018) evaluated several architectures on 87,848 images and confirmed that very high accuracy is achievable on curated data. Too et al. (2019) compared fine-tuned networks on the 38 PlantVillage classes and found that modern architectures reach above 99 % test accuracy.

Broader surveys place these results in context. Kamilaris & Prenafeta-Boldú (2018) reviewed deep learning across agricultural problems and found that it generally

outperforms conventional image processing, while noting the dependence on large labelled datasets. Liakos et al. (2018) reviewed machine learning in agriculture across crop, livestock, water and soil management, with disease detection one of the most active crop applications.

2.5.2 LIGHTWEIGHT NETWORKS

Running a classifier near the robot favours networks designed for limited compute. Sandler et al. (2018) introduced MobileNetV2, built from inverted residual blocks with linear bottlenecks, which reaches competitive accuracy at a fraction of the computation of larger networks. A MobileNetV2 fine-tuned on the 38 PlantVillage classes is used in this project; it classifies an image in tens of milliseconds on an ordinary laptop processor.

2.5.3 THE CLOSED-SET PROBLEM

A classifier trained on a fixed set of classes is closed-set: its output layer distributes probability only among those classes, so it always names one of them. Scheirer et al. (2013) formalised open-set recognition, in which inputs from unknown classes must be detected rather than forced into a known class, and showed that conventional classifiers handle this poorly. Hendrycks & Gimpel (2017) proposed using the maximum softmax probability as a simple detector of out-of-distribution inputs, noting that it helps but can be defeated, since networks often assign high confidence to unfamiliar inputs. This is exactly the failure observed in this project, where a room scene was labelled as a diseased orange leaf with 82–90 % confidence. Colour indices from vegetation imaging offer a cheap, domain-specific complement: Woebbecke et al. (1995) showed that indices such as excess green separate plant material from soil and residue reliably under varied lighting. The vegetation check in Section 6.6 builds on this idea.

2.6 PHYSICS SIMULATION FOR LEGGED ROBOTS

Simulation lets a gait be developed and tested without risking hardware. Koenig & Howard (2004) described Gazebo, an open-source simulator for multiple robots that became closely tied to the Robot Operating System. ROS 2, the current generation of that middleware, is described by Macenski et al. (2022). Todorov et al. (2012) introduced MuJoCo, a physics engine designed for model-based control, with a contact

model and integrators that are stable and fast enough for legged robots and for learning-based control. MuJoCo is now widely used in locomotion research, including work on simulation-to-reality transfer such as that of Hwangbo et al. (2019). Both simulators were used in this project, with very different outcomes, as described in Chapter 7.

2.7 SUMMARY OF THE LITERATURE

Table 2.1: Summary of surveyed work and its relevance to this project

Work	**Contribution**	**Limitation for this project**
Bledt et al. (2018)	MIT Cheetah 3: proprioceptive actuators and predictive control	Custom actuators; no sensing payload; high cost
Hutter et al. (2016)	ANYmal: robust torque-controlled field quadruped	Industrial cost and size
Katz et al. (2019); Kau et al. (2019)	Smaller, cheaper research quadrupeds	Still use brushless motors; agility rather than sensing
Hwangbo et al. (2019)	Policies learned in simulation transferred to hardware	Needs accurate simulation and capable actuators
Bechar & Vigneault (2016); Fountas et al. (2020)	Concepts and survey of field robots	Mostly wheeled; focus on harvesting and weeding
Ray (2017); Elijah et al. (2018); Tzounis et al. (2017)	IoT architectures and challenges in agriculture	Fixed sensor nodes; spatial gaps
Martinelli et al. (2015)	Review of disease detection methods including VOCs	Laboratory methods; low-cost sensors cannot identify compounds
Mohanty et al. (2016); Ferentinos (2018); Too et al. (2019)	CNNs reach >99 % on PlantVillage	Curated images; accuracy drops in the field; closed-set
Sandler et al. (2018)	MobileNetV2 for limited compute	Classification only; no notion of unknown inputs
Scheirer et al. (2013); Hendrycks & Gimpel (2017)	Open-set recognition and out-of-distribution detection	General methods; not tailored to leaves
Woebbecke et al. (1995)	Colour indices separate vegetation from background	Detects plant material, not disease
Todorov et al. (2012); Koenig & Howard (2004)	MuJoCo and Gazebo simulators	Model quality decides how far results transfer

2.8 RESEARCH GAPS

The survey identifies four gaps that this project addresses:

1. Low-cost legged platforms have not been combined with an environmental and imaging payload for crop monitoring, and their feasibility with hobby servos is rarely established quantitatively.
2. Agricultural sensing is dominated by fixed nodes; a mobile, low-cost platform that logs gas, dust and temperature along the rows has not been demonstrated at this cost level.
3. Disease classifiers trained on curated leaf images are closed-set and will report a disease for any input. A simple, explainable guard against non-leaf input suited to a field camera is needed.
4. Published hobby quadruped designs give little guidance on design factors such as the location of the centre of mass and the speed limit imposed by servo torque, which decide whether a gait will work at all.

Chapter 3
System Requirements and Design

3.1 INTRODUCTION

This chapter states what the system must do, lists the hardware and software used to build it, and explains how the architecture changed between the plan presented at Review III and the system that was finally built. The change is significant and is discussed in detail, because it follows directly from analysis carried out during the project.

3.2 REQUIREMENTS

3.2.1 FUNCTIONAL REQUIREMENTS

1. Measure total volatile organic compounds (TVOC) and estimated CO_2 (eCO_2) once per second, temperature every two seconds, particle count over a 30-second window, and the range to obstacles ahead and to the crop below.
2. Display current readings on the robot and raise an audible alert when air quality crosses a threshold.
3. Send all readings to a base station over Wi-Fi at least every five seconds, and log them permanently.
4. Stream live video from a camera on the robot to the base station.
5. On request, capture a frame, decide whether it contains a leaf, and if so classify the crop and its condition; tag the saved image with the air readings at capture time.
6. Present readings, charts, video and classification results on a web dashboard viewable from any device on the local network.
7. Provide leg kinematics and a gait that a twelve-servo quadruped can execute within the torque limits of its servos.

3.2.2 NON-FUNCTIONAL REQUIREMENTS

- **Cost:** the additional components beyond the printed chassis should cost no more than a few thousand rupees.
- **Robustness:** a missing sensor, a dropped Wi-Fi connection or a change of IP address must not stop the system or produce false readings.

- **Safety of hardware:** no expensive component should be exposed to a fall, and no 5 V signal may reach a 3.3 V input.
- **Offline operation:** the system must work without internet access at the demonstration site once set up.
- **Reproducibility:** all firmware, software, print files and simulation code are kept in a version-controlled public repository with instructions.

3.3 HARDWARE COMPONENTS

Table 3.1: Hardware used in the final system

Component	Role	Key specification
Seeed XIAO ESP32-S3 + expansion board	Sensor node controller	Dual-core 240 MHz, Wi-Fi 2.4 GHz, OLED, buzzer, Grove ports
Sensirion SGP30	TVOC and eCO_2	I^2C 0x58; 1 Hz sampling; 15 s warm-up
Microchip MCP9808	Air temperature	I^2C 0x18; ±0.25 °C typical; 0.0625 °C resolution
Shinyei PPD42	Particle count (>1 μm)	5 V; low-pulse occupancy output; 30 s window
HC-SR04 × 2	Front and downward range	5 V; 2–400 cm; echo pulse output
ESP32-CAM (AI-Thinker) + MB board	Camera node	OV3660 sensor; 4 MB PSRAM; USB programming via CH340
Grove Relay v1.3	Actuation output (planned)	Switches a 5 V pump or alarm
12 × MG996R servos	Leg actuation	0.92–1.08 N·m stall at 4.8–6 V
Printed chassis	Structure	35 parts, PLA+ with TPU feet
Laptop	Base station	Wi-Fi hotspot, dashboard, classifier

3.4 SOFTWARE COMPONENTS

Table 3.2: Software and tools

Layer	Tools
Embedded firmware	Arduino core for ESP32 (3.3.12), Adafruit SGP30 library, U8g2 display library, ESP-IDF HTTP server; built and flashed with arduino-cli
Dashboard	Python 3.11, Flask (Grinberg, 2018), Requests, Pillow; HTML, CSS and JavaScript with no external libraries
Classification	PyTorch and Hugging Face Transformers running a MobileNetV2 checkpoint fine-tuned on PlantVillage

Layer	Tools
Simulation	MuJoCo 3.15 Python bindings, NumPy, Matplotlib, imageio for video; earlier work in ROS 2 Humble with Gazebo Classic 11 and a browser simulator in Three.js
Mechanical design	SpotMicro STL files; Python with trimesh and manifold3d to generate and verify the custom parts
Collaboration	Git and GitHub

3.5 INITIAL ARCHITECTURE

The architecture proposed in the early reviews is shown in Figure 3.1. It placed an NVIDIA Jetson Orin Nano on the robot as the compute core, running crop disease detection, environmental data processing, path planning, the gait controller and localisation. The perception layer included an IMX219 camera, a PMS5003 dust sensor, an MQ-135 gas sensor, an IMU, a GPS module and an optional LiDAR, and the communication layer offered Wi-Fi, LoRa and Bluetooth links to a base station.

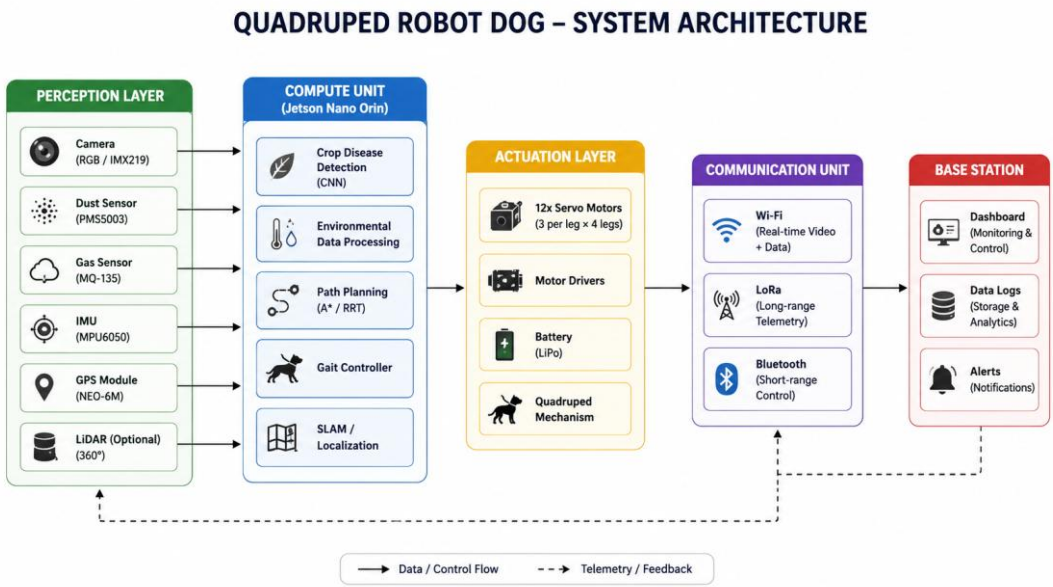


Figure 3.1: System architecture proposed in the early project reviews

By Review III the sensor choices had already been refined: the analogue MQ-135 was replaced by the digital SGP30, which removed the need for an analogue-to-digital converter, and the PPD42 replaced the PMS5003 because it was in hand. The Jetson remained the central element.

3.6 DESIGN REVISION AND RATIONALE

Work after Review III exposed several problems with an onboard Jetson. The Orin Nano requires a 9–19 V supply, not the 5 V assumed in the first power design, and dissipates 7–25 W, enough to soften the PLA chassis, which loses stiffness around 60 °C. The static torque analysis in Section 4.5 showed that the knee servos would operate at up to 70 % of their rated torque in a crawl gait even before the Jetson's mass was considered, leaving little margin for the dynamic loads of walking. A fall during walking trials would put the most expensive component of the project at risk. Finally, the ESP32-S3 selected for the sensors has no MIPI-CSI camera interface, so the IMX219 camera could not be moved to a microcontroller.

The revised design separates sensing from computation. Two inexpensive microcontroller nodes ride on the robot and send their data over Wi-Fi to a laptop, which runs the dashboard and the classifier. Inference on a laptop needs no onboard GPU, the robot becomes lighter, and nothing of significant value is exposed to a fall. Locomotion, which the torque analysis showed to be the hardest part of the project, was moved into physics simulation so that it could be developed and evaluated quantitatively before risking hardware. Table 3.3 summarises the changes.

Table 3.3: Changes between the Review III plan and the final system

Review III plan	**Final system**	**Reason**
Jetson Orin Nano on the robot	XIAO ESP32-S3 and ESP32-CAM nodes; inference on a laptop	Supply, heat and fall risk; thin torque margin; no onboard GPU needed
IMX219 CSI camera	ESP32-CAM with OV3660, MJPEG over Wi-Fi	ESP32-S3 has no MIPI-CSI interface
ROS 2 and MQTT on the robot	HTTP and JSON to a Flask dashboard	Fewer components; all devices on one local network
Gazebo dynamics model	MuJoCo physics model	Gazebo stance never stabilised; MuJoCo walks and reports metrics
Walking trials on hardware	Gait developed in simulation; legs actuated and standing tested on hardware	Simulation showed a centre-of-mass correction is needed first
3S LiPo and onboard regulators	USB power bank for the nodes; mains supply for servo tests	Safer and simpler for bench work

3.7 FINAL SYSTEM ARCHITECTURE

Figure 3.2 shows the system as built. The sensor node and the camera node both join a 2.4 GHz Wi-Fi hotspot hosted by the laptop, which always takes the address 192.168.137.1 on its own hotspot. The sensor node posts a JSON record of all its readings every five seconds. The camera node streams Motion-JPEG video and, at boot, announces its own address to the dashboard. The laptop runs the Flask dashboard, stores the readings and images, and runs the classifier on demand. Any phone or laptop on the same hotspot can open the dashboard in a browser.

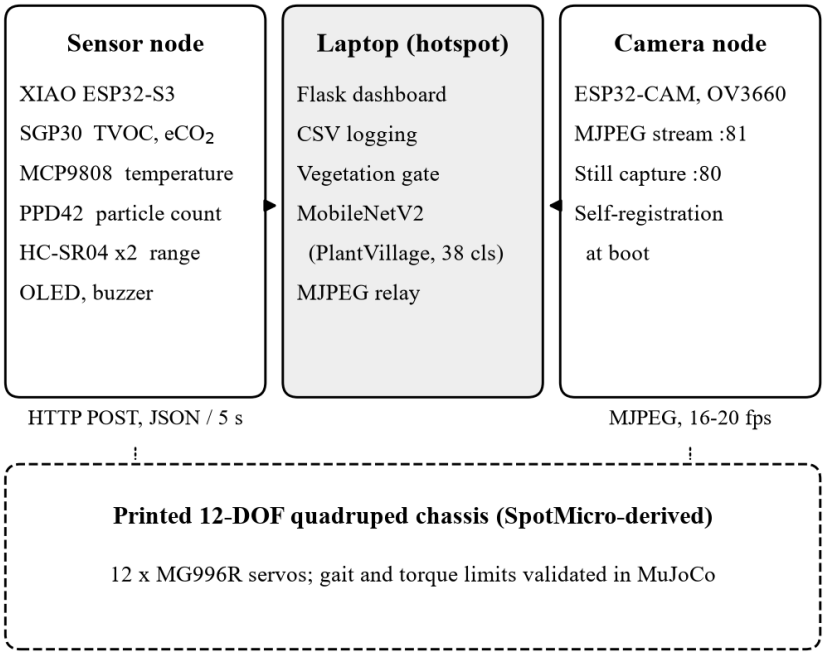


Figure 3.2: Final system architecture: two sensing nodes on the chassis and a laptop base station on a shared Wi-Fi hotspot

3.8 COMMUNICATION DESIGN

All communication uses HTTP on the local network, which keeps every component simple to test with a browser or a command-line client. Table 3.4 lists the message flows. The sensor node pushes data rather than waiting to be polled, so the dashboard does not need to know its address. The camera node is pulled for video, so it must be known; its self-registration at boot removes the need to look its address up, which matters because the hotspot may assign a different address each time.

Table 3.4: Message flows between the components

Flow	Method	Content
Sensor node → dashboard	POST /api/sensors	JSON record of all readings, every 5 s
Camera node → dashboard	POST /api/camera	Its still-image and stream URLs, at boot and after any address change
Dashboard → camera node	GET :81/stream	One continuous MJPEG connection
Browser → dashboard	GET /api/state	Latest readings, ten-minute history, camera and classifier status; polled every 2 s
Browser → dashboard	GET /api/camera/stream	Relayed MJPEG video
Browser → dashboard	POST /api/scan, /api/classify	Classify the current frame or an uploaded image

3.9 SUMMARY

The requirements emphasise robustness, low cost and safety of hardware as much as function. The design that meets them differs from the Review III plan: sensing is distributed across two microcontrollers, computation moves to a laptop, and locomotion is developed in simulation first. The following chapters describe each part in turn.

Chapter 4
Mechanical Design and Fabrication

4.1 BASE PLATFORM

The chassis is derived from SpotMicro, an open quadruped design published by the user KDY0523 under a Creative Commons Attribution licence (KDY0523, n.d.). SpotMicro was chosen because it is designed for standard-size hobby servos, prints without support material on most parts, and has an active community that has documented its assembly. Each leg has three joints: an abduction joint that swings the leg sideways, a hip joint and a knee joint. All twelve joints are driven directly by servos, with the abduction servos housed in the body and the hip and knee servos in the shoulder and upper leg.

4.2 FABRICATION

The body and leg parts were printed in PLA+ and the foot pads in TPU, which gives grip and absorbs impact. The complete set is 35 parts, collected into a single print-ready folder with quantities in the file names (Figure 4.1). The structural parts used about 700 g of PLA+ and the foot pads about 12 g of TPU; printing took 30–40 hours and cost ₹5,200. Parts were checked for warping and fit before assembly.

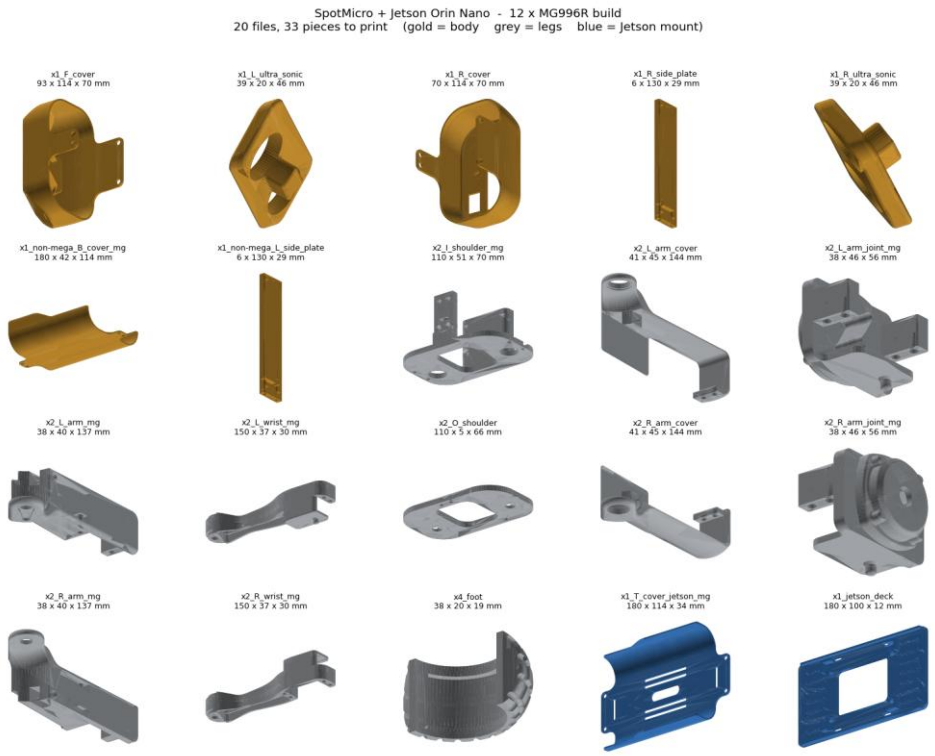


Figure 4.1: The print-ready part set: body parts (gold), leg parts (grey) and the team-designed mounting parts (blue)

4.3 MEASURED GEOMETRY

The kinematic model and the simulation both depend on the leg geometry, so the dimensions were measured from the printed parts rather than taken from drawings. The lateral spacing of the abduction axes was taken from the centres of the 16.3 mm bearing seats in the printed shoulder plates, which are the physical pivot points. Table 4.1 lists the values used throughout the report.

Table 4.1: Geometry measured from the printed parts

Parameter	**Value**	**Source**
Hip spacing, longitudinal × lateral	140 × 78 mm	Bearing seats in the printed body
Shoulder offset, abduction axis to leg plane (L_0)	60 mm	Printed shoulder bracket
Femur length (L_1)	111 mm	Printed upper leg
Tibia length (L_2)	125 mm	Printed lower leg
Mass with Jetson and battery (Review III)	1.757 kg	Print volumes and component weights
Mass, final configuration	1.35 kg	Without onboard computer or battery

4.4 TEAM-DESIGNED PARTS

The stock chassis was designed around an Arduino Mega and has no place for the payload this project carries. Four parts were designed by the team to mount it, generated parametrically in Python so that their dimensions could be changed and the parts regenerated quickly:

- **Payload deck** (`jetson_deck`): an external deck that carried the Orin Nano in open air in the Review III design, and now provides a flat mounting surface on top of the body.
- **Modified top cover** (`T_cover_jetson_mg`): the stock cover with slots for camera and wiring harness and vents for airflow.
- **PPD42 bracket**: holds the dust sensor upright, which its internal heater requires to drive air through the sensing chamber by convection.
- **SGP30 bracket**: raises the gas sensor 22 mm into clear air, away from the body.

The external deck was needed because the Orin Nano development kit measures 103 × 90.5 × 34.7 mm, while the internal bay is 107 × 95 mm and also had to hold the battery and power electronics; enclosing a 7–25 W board in PLA would also have risked softening the shell. All four parts were verified as watertight solid meshes with no defective edges, and Boolean intersection with the deck and the board envelope confirmed zero interference volume.

4.5 STATIC TORQUE ANALYSIS

Whether hobby servos can hold up the robot was checked before any walking was attempted. A quasi-static model computed the load on each foot from force and moment balance, then the torque at each joint as the component of the moment of the foot force about the joint axis:

$$\tau = |(\mathbf{r} \times \mathbf{F}) \cdot \mathbf{a}| \tag{4.1}$$

where $\mathbf{r}$ is the vector from the joint to the foot, $\mathbf{F}$ the foot contact force and $\mathbf{a}$ the unit joint axis. Table 4.2 shows the results for the Review III mass of 1.757 kg.

Table 4.2: Static knee torque against the MG996R rating (Review III mass, 1.757 kg)

Condition	Foot load	Knee torque	Share of 11 kg·cm
Four-foot stance	0.44 kgf	4.0 kg·cm	36 %
Four-foot stance, stride offset	0.44 kgf	5.7 kg·cm	52 %
Three-foot crawl support	0.59 kgf	7.65 kg·cm	70 %

The knee is the critical joint in every case. The hip torque stays close to zero when the foot is directly below the hip, because the foot force then passes almost through the hip axis. The peak knee torque of 4.8 kg·cm found in a four-foot stance simulation agreed with the analytical range of 4.0–5.7 kg·cm, which gave confidence in both. The margin in the crawl case is about 30 % for genuine MG996R servos, which are rated at about 11 kg·cm at 6 V (Tower Pro, n.d.), but many inexpensive copies deliver only 7–9 kg·cm, which they would exceed. This finding fed directly into the design revision of Chapter 3: removing the Jetson and the battery from the robot reduces the mass to about 1.35 kg and the torque demand in proportion.

4.6 ASSEMBLED ROBOT

Figure 4.2 shows the assembled robot standing on its own legs outdoors. The sensor node and its breadboard sit on the printed top cover, with the node's external Wi-Fi antenna raised on a standoff, and the ESP32-CAM looks out through the opening in the printed head. The payload is powered over USB from a power bank beside the robot, so no battery or regulator is carried on the chassis.

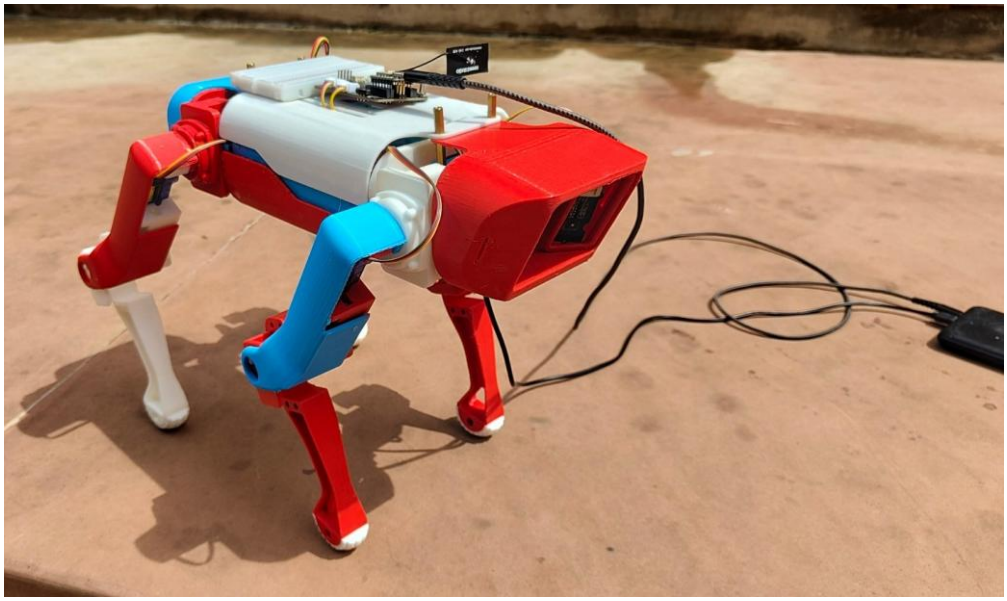


Figure 4.2: The assembled robot: sensor node and antenna on the top cover, camera node in the head, payload powered from a USB power bank

4.7 SUMMARY

The chassis was printed, its geometry measured, and four parts designed to carry the payload. The static analysis identified the knee servos as the binding constraint and quantified the margin, which shaped both the decision to lighten the robot and the use of simulation to develop the gait.

Chapter 5
Embedded Sensing System

5.1 INTRODUCTION

The sensing payload consists of two microcontroller nodes: a sensor node that measures air quality, temperature and range, and a camera node that captures images. This chapter describes the hardware of each, the electrical interfacing between 5 V sensors and 3.3 V microcontrollers, and the firmware, with particular attention to the design decisions that came from testing.

5.2 SENSOR NODE HARDWARE

The sensor node is built on a Seeed XIAO ESP32-S3, a thumb-sized module with a dual-core 240 MHz processor, 8 MB of PSRAM and 2.4 GHz Wi-Fi (Espressif Systems, n.d.). It plugs into the Seeed XIAO expansion board, which adds a 128 × 64 pixel OLED display, a passive buzzer, a push button, a real-time clock, a microSD slot and Grove connectors for I²C, UART and analogue signals. The expansion board removed most of the wiring that a bare module would have needed, and the screen and buzzer give the robot a local interface independent of the dashboard (Figure 5.1).

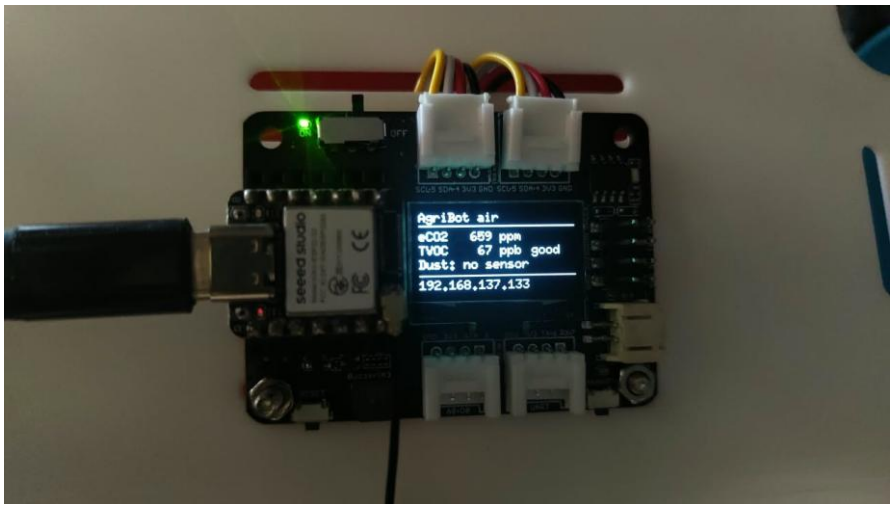


Figure 5.1: Sensor node on the XIAO expansion board, showing live gas readings and its network address on the OLED

Two sensors connect through the board's Grove I²C ports: the SGP30 gas sensor and the MCP9808 temperature sensor. The PPD42 dust sensor and the two HC-SR04 ultrasonic sensors use digital pins through a breadboard. Table 5.1 gives the full pin allocation. The microSD slot shares pins D8–D10 with its SPI bus; since every reading

is already logged on the laptop, the slot is left empty and those pins drive the second ultrasonic sensor.

Table 5.1: Sensor node pin allocation

Pin	Function	Notes
D0	PPD42 particle output	Through voltage divider; pull-down enabled
D1	Push button (mute)	On the expansion board
D2	Free	microSD chip select, unused
D3	Buzzer	On the expansion board
D4, D5	I²C SDA, SCL	OLED 0x3C, MCP9808 0x18, RTC 0x51, SGP30 0x58
D6, D7	Front ultrasonic trigger, echo	Echo through voltage divider
D8, D9	Downward ultrasonic trigger, echo	Echo through voltage divider
D10	Free	Reserved for a soil-moisture probe

5.3 ELECTRICAL INTERFACING

5.3.1 VOLTAGE DIVIDERS

The XIAO's inputs tolerate at most 3.3 V, but the PPD42 output swings to about 4 V and the HC-SR04 echo pulse to 5 V. Each of these signals therefore passes through a resistive divider. With a top resistor R_1 and a bottom resistor R_2, the pin sees

$$V_{pin} = V_{in} \cdot R_2 / (R_1 + R_2) \tag{5.1}$$

With R_1 = 1 kΩ and R_2 = 2 kΩ, a 5 V echo becomes 3.33 V and the PPD42's 4 V output becomes 2.67 V, which is still well above the ESP32-S3's logic-high threshold. The 2 kΩ is made from two 1 kΩ resistors in series, so that the whole node needs only one resistor value. The wire to the microcontroller must be taken from the junction between R_1 and R_2; taking it from the sensor side of R_1 would apply the full 5 V to the pin.

5.3.2 I²C BUS AND POWER

The OLED, the real-time clock, the MCP9808 and the SGP30 share one I²C bus. Their addresses (0x3C, 0x51, 0x18 and 0x58) are all distinct, and a PCA9685 servo driver at 0x40 can be added later without conflict. The I²C devices run from the board's 3.3 V supply. The PPD42 and ultrasonic sensors need 5 V, which is taken from the

expansion board's 5 V pin onto the breadboard power rail; together they draw about 120 mA, well within the capacity of a USB supply.

5.3.3 WIRING

A complete wiring document was produced for assembly and is reproduced in Appendix 1. Figure 5.2 shows its schematic: the Grove sensors connect directly to the board, while each 5 V output passes through its divider on the breadboard. The document also gives a breadboard layout with every resistor position, a wire-by-wire checklist and a recommended order for adding sensors one at a time.

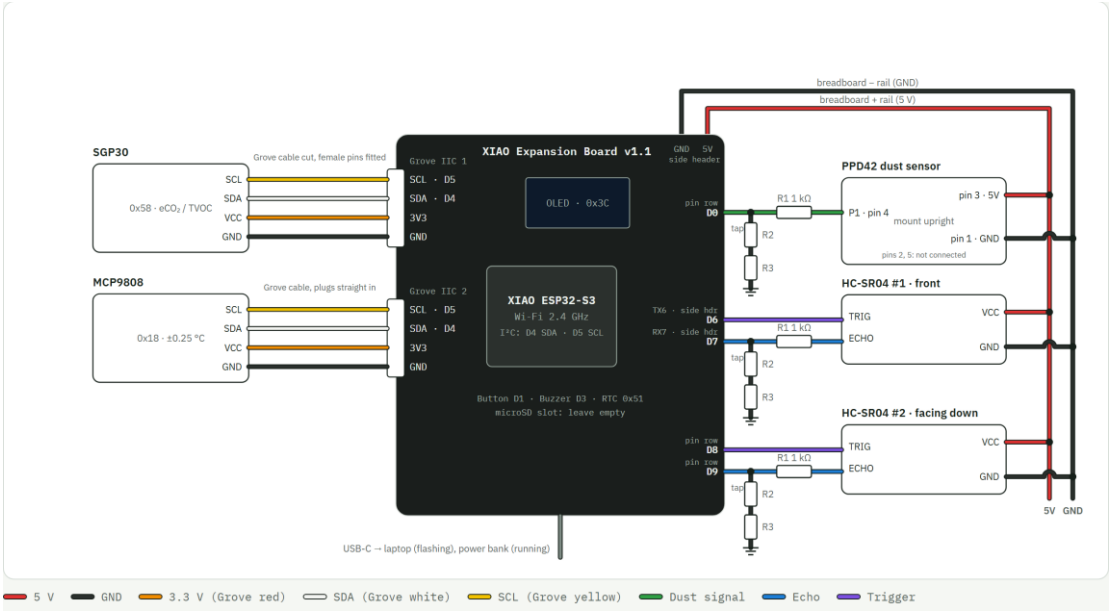


Figure 5.2: Wiring schematic of the sensor node; every 5 V sensor output reaches the XIAO through a divider

5.4 SENSOR NODE FIRMWARE

5.4.1 STRUCTURE AND SCHEDULING

The firmware is written in C++ on the Arduino core for the ESP32. Instead of a real-time operating system it uses a cooperative loop in which each task runs when its own period has elapsed, measured against the millisecond clock. The only time-critical signal, the PPD42 output, is handled by an interrupt, so no task needs to wait for it. Figure 5.3 shows the resulting schedule and Table 5.2 the periods.

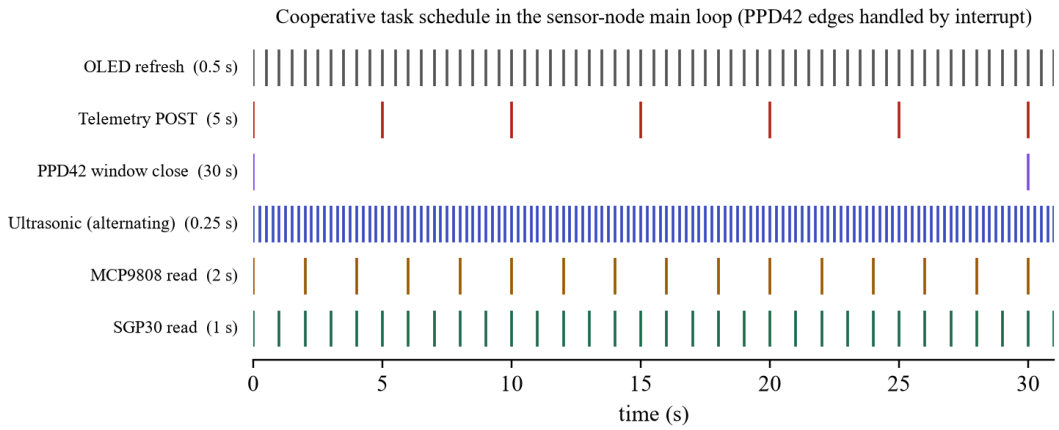


Figure 5.3: Task schedule of the sensor node over one 30-second dust window

Table 5.2: Firmware task periods

Task	**Period**	**Reason**
SGP30 measurement	1 s	The sensor's baseline compensation expects 1 Hz
MCP9808 measurement	2 s	Temperature changes slowly
Ultrasonic ranging	0.25 s	Alternates sensors, so each runs at 2 Hz
PPD42 window	30 s	Manufacturer's recommended sampling window
Telemetry to dashboard	5 s	Matches the dashboard's display rate
OLED refresh	0.5 s	Two pages alternate every 4 s
Wi-Fi supervision	10 s	Retry only after an attempt has ended

5.4.2 GAS SENSING

The SGP30 is a metal-oxide sensor with several sensing elements on one chip. It reports total VOC in parts per billion and an estimated CO_2 equivalent in parts per million derived from its hydrogen signal (Sensirion AG, n.d.). For its first 15 seconds after power-up it returns fixed values of 400 ppm and 0 ppb, so the firmware marks this period as warm-up. Readings are graded against two thresholds per quantity: TVOC of 220 and 660 ppb, and eCO_2 of 1,000 and 2,000 ppm, dividing the air into good, moderate and high. When the grade first becomes high, the buzzer sounds three short beeps; the push button mutes it.

5.4.3 DUST SENSING

The PPD42 passes air through a light beam and pulls its output low while particles cross it. The fraction of time the output is low over a sampling window, the

low-pulse occupancy, rises with particle concentration (Shinyei Technology, n.d.). A common way to measure this is a blocking pulse-timing call that stops the program for up to the length of each pulse. Here an interrupt fires on every change of the output, recording the start of each low period and accumulating its length. At the end of each 30-second window the accumulated low time is read and reset in a critical section, and a pulse still in progress is split at the boundary so that no time is lost or counted twice. With r the low time as a percentage of the window, the manufacturer's curve gives the particle count C per 0.01 ft³:

$$C = 1.1r^3 - 3.8r^2 + 520r + 0.62 \tag{5.2}$$

Testing exposed a serious flaw. With the PPD42 disconnected, its input floated or was held low by the divider, and the firmware read a line that was low for the whole window. Equation (5.2) turned this into about 1.1 million particles, the grade became high and the buzzer sounded. A working sensor idles high and only pulses low, so a window with no edges at all and a low line cannot come from a real sensor. The firmware now counts edges in the interrupt, enables an internal pull-down on the pin so an unconnected input reads low rather than noise, and reports such a window as an absent sensor, without an alarm. The fix was verified on the hardware, which then printed a clear diagnostic instead of a false count.

5.4.4 TEMPERATURE MEASUREMENT

The MCP9808 is read directly over I²C without a library. At start-up the firmware reads the manufacturer and device identification registers and accepts the sensor only if they match, which guards against a different device at the same address. The ambient temperature register holds a 13-bit two's-complement value in units of 0.0625 °C (Microchip Technology Inc., n.d.).

5.4.5 ULTRASONIC RANGING

Each HC-SR04 is triggered with a 10 µs pulse and the length of its echo pulse is measured with a 25 ms timeout, which corresponds to about 4.3 m, the sensor's limit. Distance follows from the speed of sound, which depends on air temperature:

$$c = 331.3 + 0.606T \ \text{m/s}, \quad d = c \cdot t_{\text{echo}} / 2 \tag{5.3}$$

where T is the air temperature in °C, taken from the MCP9808. On a 35 °C day the usual fixed value of 343 m/s would underestimate the speed of sound by about 3 %, and

every distance would be short by the same proportion. The two sensors are never fired together, so that neither receives the other's pulse, and each reported value is the median of the last three readings, which rejects isolated false echoes. The front sensor flags an obstacle below 25 cm. The downward sensor reports canopy height as the difference between its mounting height and the measured distance.

5.4.6 DISPLAY, TELEMETRY AND WI-FI

The OLED alternates every four seconds between an air page (eCO_2, TVOC and dust with their grades) and a field page (temperature, front distance with a STOP flag, and canopy height), with the node's IP address always shown at the bottom. Every five seconds the node sends a JSON record of all readings to the dashboard; the format is given in Appendix 2. Readings that are unavailable, such as a range with no echo, are sent as null rather than as a misleading number.

Wi-Fi supervision proved important. During bring-up the node could not join the hotspot while the camera node, on the same desk with the same credentials, joined immediately. The firmware at first reported only that it was connecting. A short diagnostic program scanned for networks and logged the reason for each failed attempt: the hotspot was heard at only −78 dBm, and every attempt failed with an authentication time-out. The XIAO ESP32-S3 has no built-in antenna, and its external antenna had not been attached; once fitted, the node heard the hotspot at −26 dBm and connected at once. The firmware now reports the reason for a failure, such as network not found or connection failed, and retries only after an attempt has finished, because restarting a connection while the driver is still negotiating aborts it.

5.5 CAMERA NODE

5.5.1 HARDWARE

The camera node is an AI-Thinker ESP32-CAM: an ESP32 module with 4 MB of PSRAM and a camera connector, fitted here with a 3-megapixel OV3660 sensor. It sits on an ESP32-CAM-MB adapter that provides a USB-to-serial converter and the automatic reset needed for programming. Before flashing, the module's factory firmware was used to confirm that the camera worked: it served a 640 × 480 image of correct exposure over its own Wi-Fi network, and its settings showed the colour and orientation corrections that the driver applies only when it detects the OV3660.

5.5.2 FIRMWARE

The camera captures 640 × 480 JPEG frames at quality 12, about 11–18 KB each, using two frame buffers in PSRAM. The driver is set to always return the newest frame, because its default mode can return a frame captured seconds before a request, which would make a scan show the leaf that was in view earlier rather than the one being held up. Two HTTP servers run on separate ports. The server on port 81 streams Motion-JPEG continuously; the one on port 80 returns single frames and a status record. Because each server runs in its own task, a capture never waits for a stream to end, and the main loop remains free to supervise Wi-Fi.

5.5.3 SELF-REGISTRATION

The laptop hotspot assigns addresses dynamically, and the camera received different addresses on different days. Rather than require the user to find the address and type it into the dashboard, the camera posts its still-image and stream URLs to the dashboard when it connects, retrying every 15 seconds until the dashboard accepts them, and again whenever its address changes. In testing, the camera appeared on the dashboard within seconds of power-up at every boot, including after its address changed.

5.6 DEVELOPMENT WORKFLOW

Both firmwares were built and flashed with arduino-cli using the Espressif ESP32 core 3.3.12. The sensor node firmware uses 31 % of the XIAO's program flash and 15 % of its RAM; the camera firmware uses 36 % of the ESP32-CAM's flash. After every change the firmware was compiled with warnings enabled, flashed over USB, and verified by reading the serial output, which reports each sensor found, each dust window and a summary of the readings every five seconds. Wi-Fi credentials are kept in a separate header file excluded from version control, so that the public repository never contains them.

5.7 SUMMARY

The two nodes provide gas, temperature, dust, range and image data over Wi-Fi from inexpensive hardware. Several firmware features came directly from faults found in testing: the absent-sensor check, the Wi-Fi diagnostics and the camera's self-

registration. Each addresses a failure that would otherwise appear as wrong data or a silent stall during a demonstration or in the field.

Chapter 6
Dashboard and Leaf-Disease Classification

6.1 INTRODUCTION

The laptop is the system's base station. It runs a web dashboard that receives and stores the sensor data, relays the camera's video and classifies leaf images. This chapter describes the dashboard's architecture and interface, the video pipeline, the classification model and the vegetation check that protects it from non-leaf input.

6.2 DASHBOARD ARCHITECTURE

The dashboard is a Flask application (Grinberg, 2018). Figure 6.1 shows its structure. Route handlers receive sensor records and camera registrations, serve the current state as JSON and accept scan and upload requests. A background thread holds the connection to the camera, and the classifier loads in another thread at start-up so that the dashboard is usable immediately. All shared state is protected by a lock.

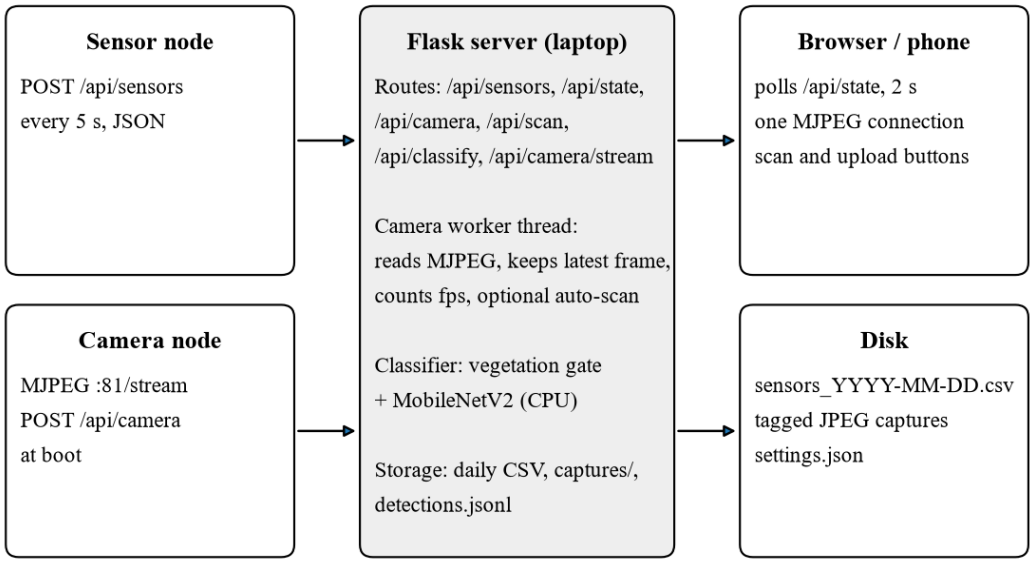


Figure 6.1: Software architecture of the dashboard

Sensor records are kept in memory for the last hour, of which the charts show the last ten minutes, and appended to a CSV file per day. The sensor node is considered offline if no record has arrived for 15 seconds, three missed posts. The camera settings are saved to disk, so a restarted dashboard reconnects to the camera without waiting for it to register again. The interface is a single HTML page with its own styles and scripts and no external libraries, so it works without internet access.

6.3 USER INTERFACE

Figure 6.2 shows the upper part of the dashboard during a test session. Status indicators at the top show whether the sensor node, the camera and the classifier are working. Four cards show eCO_2, TVOC, dust and temperature, each with its grade, the sensor it comes from and a chart of the last minute. In the figure all four sensors are connected and reporting, with the gas and dust readings in the good band. The ultrasonic readings, which the robot needs only for navigation, are not shown on the dashboard. Below the cards, the camera panel shows live video with its frame rate and offers buttons to scan the current frame or upload an image, and the diagnosis panel shows the latest result. A gallery at the bottom shows recent tagged captures.

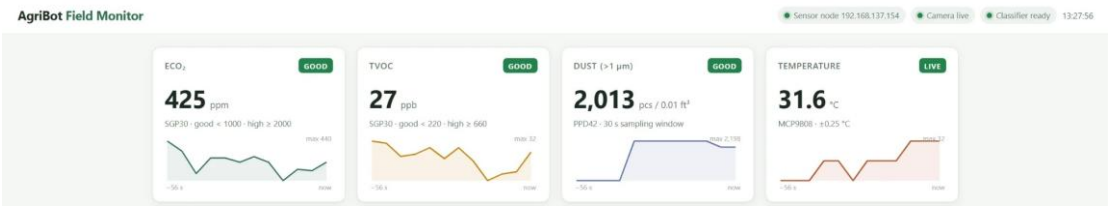


Figure 6.2: Dashboard on 18 October 2026, with live gas, dust and temperature readings from the physical sensor node

6.4 VIDEO PIPELINE

The first version of the camera pipeline fetched a single still image every second, and the page reloaded it every two seconds, so the user saw about one new frame every two seconds. This made it hard to hold a leaf steady in view. The pipeline was rebuilt around streaming. The dashboard's camera thread opens one long-lived connection to the camera's Motion-JPEG stream and extracts each JPEG by locating its start and end markers, which works with any camera that serves such a stream. Each new frame is stored and a condition variable wakes every browser connection, which receives the frame as part of its own Motion-JPEG response.

Because the camera serves only one stream, to the dashboard, any number of viewers can watch without loading the ESP32-CAM. The frame rate is measured by counting frames over a two-second window; an earlier estimate based on the gap between consecutive frames overstated the rate badly, because frames arrive over Wi-Fi in bursts. The measured rate after the change was 16–20 frames per second. Cameras without a stream, such as a phone running an IP-webcam app, are handled by polling still images instead.

6.5 CLASSIFICATION MODEL

The classifier is a MobileNetV2 network (Sandler et al., 2018) fine-tuned on the 38 classes of the PlantVillage dataset (Hughes & Salathé, 2015), covering healthy and diseased leaves of 14 crops including apple, corn, grape, potato, pepper and tomato. A publicly available fine-tuned checkpoint is used without further training and runs on the laptop's processor through the Hugging Face Transformers library. The class names in the checkpoint come in several forms, such as "Tomato with Early Blight", "Healthy Tomato Plant" and "Apple Scab", so the dashboard identifies the crop by name rather than by position and splits each label into a crop and a condition for display. A result with top probability below 0.5 is reported as uncertain.

6.6 VEGETATION GATE

During testing the camera was pointed at an empty room and a scan was taken. The classifier returned "orange with citrus greening" with 89.7 % confidence, and a second scan of a similar scene returned the same label at 82 %. As discussed in Chapter 2, a closed-set classifier must assign one of its classes to any input, and its confidence offers little protection (Scheirer et al., 2013; Hendrycks & Gimpel, 2017). A false diagnosis of this kind would undermine trust in the whole system.

The vegetation gate runs before the classifier and asks a simpler question: is there enough leaf-coloured material in the frame? The frame is reduced to 160 × 120 pixels and converted to the HSV colour space. A pixel counts as leaf-coloured if its hue H, saturation S and value V satisfy

$$38° \le H \le 165°, \quad S \ge 0.20, \quad V \ge 0.12 \tag{6.1}$$

and the leaf coverage f is the fraction of pixels that do. The hue band spans green and the yellow of chlorotic leaves, so diseased leaves still pass, while skin, wood and soil fall below it. Walls and floors have low saturation and fail the second condition, and very dark pixels fail the third. A frame with f below 0.12 is reported as "no leaf in view" and the classifier's answer is not shown. The idea follows the colour indices long used to separate vegetation from background in crop imaging (Woebbecke et al., 1995).

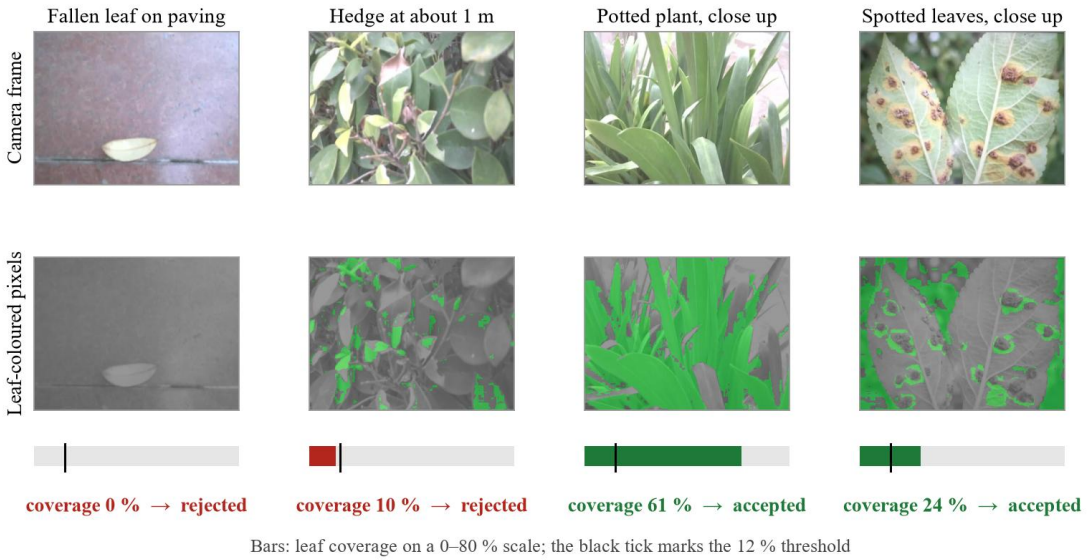


Figure 6.3: The vegetation gate on four camera frames from 18 October 2026: leaf-coloured pixels are highlighted in green, and frames below 12 % leaf coverage are rejected before classification

Figure 6.3 shows the gate on four frames from the camera. A fallen, dried leaf on paving contains no pixels in the green band and is rejected. A hedge seen from about a metre away is also rejected: its leaves are small in the frame and the camera's washed-out colours leave many of them below the saturation limit. A potted plant filling the frame and a close view of spotted leaves are both accepted. In the spotted leaves the brown lesions themselves are not counted as leaf-coloured, but the surrounding green tissue is still well above the threshold, so diseased leaves reach the classifier. The coverage is computed on the frame without the dashboard's label banner, so it differs slightly from the value shown on the dashboard.

6.7 IMAGE TAGGING AND LOGGING

Every accepted scan, and every rejected one, is saved as an image with a coloured banner stating the crop, the condition and the confidence: green for a healthy leaf, red for a disease and grey for an uncertain result or no leaf. The record also stores the sensor readings at the moment of capture, so that each leaf image carries the gas, dust and temperature conditions around it. Records are appended to a log file and shown in the dashboard's gallery. Images can also be uploaded from a file, which allows classification to continue if the camera network fails during a demonstration.

6.8 TESTING WITHOUT HARDWARE

A small program that imitates the sensor node, posting the same JSON records with slowly varying values and periodic spikes, was used to develop and test the dashboard before the hardware was ready. The MuJoCo simulation of Chapter 7 can also post its simulated readings to the dashboard, which shows the same interface displaying data from the simulated robot.

6.9 SUMMARY

The dashboard brings together data from both nodes, keeps a permanent record and presents it live. The classification pipeline combines a standard lightweight network with a simple, explainable check that removes its most damaging failure in practice, a confident disease report for a frame with no plant in it.

Chapter 7
Kinematics, Gait and Simulation

7.1 INTRODUCTION

Locomotion was the part of the project with the most uncertainty, so it was developed in stages: a closed-form kinematic model, a quasi-static simulator for checking loads, an attempt at a full dynamic simulation in Gazebo, and finally a dynamic simulation in MuJoCo that walks, turns and carries out an autonomous survey. This chapter describes each stage and what it contributed.

7.2 LEG KINEMATICS

7.2.1 COORDINATE FRAMES

The body frame has x pointing forward, y to the left and z up. Each leg's foot target is expressed relative to that leg's abduction joint, with a side sign $s = +1$ for left legs and $s = -1$ for right legs. The abduction joint rotates about x; the hip and knee joints rotate about the leg's own lateral axis. A positive hip angle swings the foot backward, and the knee bends to negative angles, which places the knee behind the foot as on the physical robot (Figure 7.1).

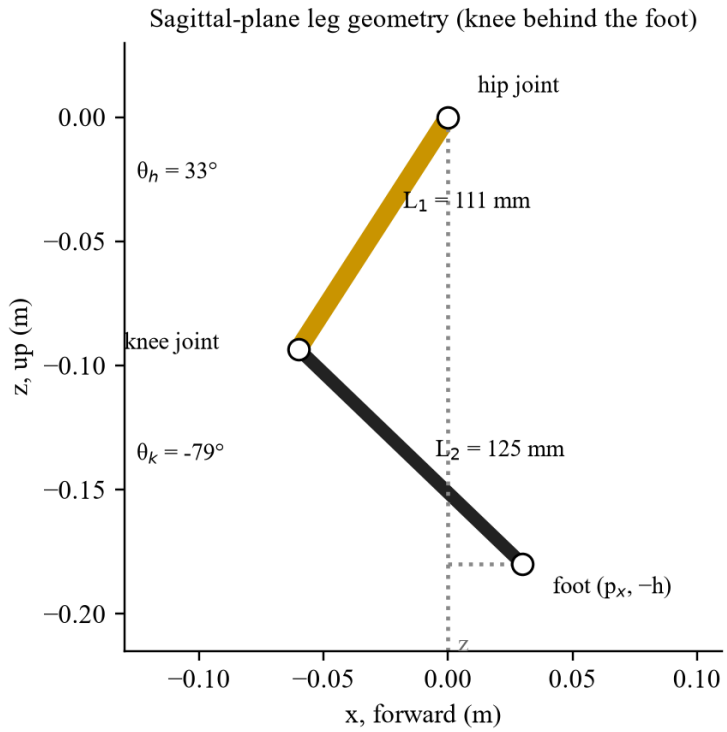


Figure 7.1: Leg geometry in the sagittal plane for a foot 30 mm ahead of the hip at a body height of 180 mm

7.2.2 CLOSED-FORM INVERSE KINEMATICS

The abduction joint sits a distance L_0 inboard of the plane in which the femur and tibia move. Projecting the foot target (p_x, p_y, p_z) onto the plane perpendicular to the abduction axis gives the effective leg length h in that plane and the abduction angle θ_a:

$$h = (p_y^2 + p_z^2 - L_0^2)^{1/2} \tag{7.1}$$

$$\theta_a = \text{atan2}(p_z, p_y) - \text{atan2}(-h, s \cdot L_0) \tag{7.2}$$

In the leg plane the foot lies at $(p_x, -h)$ relative to the hip, a standard two-link problem. The law of cosines gives the knee angle, and the hip angle follows from the direction of the foot and the angle the tibia adds to it:

$$\theta_k = -\arccos[(p_x^2 + h^2 - L_1^2 - L_2^2) / (2L_1L_2)] \tag{7.3}$$

$$\theta_h = \text{atan2}(-p_x, h) - \text{atan2}(L_2 \sin\theta_k, L_1 + L_2 \cos\theta_k) \tag{7.4}$$

Both signs of the arccos in (7.3) satisfy the link lengths, but only the negative branch puts the knee behind the foot as assembled; the positive branch folds the leg the other way. A closed-form solution was preferred over a numerical solver because it has no iterations, cannot fail to converge and costs a few microseconds per leg.

7.2.3 VERIFICATION

An early version of the solver used the wrong branch and placed feet above the hips, an error that was visible only when the legs were drawn. To prevent such errors, the final solver was checked against the forward kinematics of the MuJoCo model itself. For 1,280 foot targets spread over the working volume of all four legs, the joint angles from the solver were set in the model, MuJoCo computed where each foot ended up, and the result was compared with the target. The largest error was below 10^{-7} m, which is numerical round-off; an earlier check against an independent forward-kinematics implementation had given 5.7×10^{-17} m.

7.3 EVOLUTION OF THE SIMULATION

7.3.1 QUASI-STATIC BROWSER SIMULATOR

The first simulator ran in a web browser using the Three.js graphics library. It drew the robot from its measured geometry, moved the legs with the inverse kinematics, and solved the statics at each instant: the centre of mass, the support polygon formed by the feet on the ground, the stability margin, the distribution of weight among the feet from force and moment balance, and the resulting torque at each joint using (4.1). The torques were compared with an adjustable servo limit. This simulator produced the torque results in Table 4.2, and its peak knee torque agreed with the hand calculation. Being quasi-static, it could not show whether the robot would actually stay upright while moving.

7.3.2 GAZEBO AND ROS 2

A full dynamic simulation was then built in Gazebo Classic 11 (Koenig & Howard, 2004) under ROS 2 Humble (Macenski et al., 2022), running in Ubuntu 22.04 under the Windows Subsystem for Linux. A Python generator wrote the robot description from the measured geometry, and the joints were driven through the ros2_control framework. Several problems were solved along the way: missing plug-ins, malformed model files, controller start-up order and network configuration within the subsystem. Two problems could not be solved in the time available.

First, the position-control interface in this setup sets joint positions directly rather than applying torque, which ignores the servo limits entirely and, for a robot with a free-floating body, injected so much energy that the model was thrown hundreds of

metres. Driving the joints through the effort interface with a torque-limited PD loop was physically correct, but the controller update rate and gains could not be tuned to hold a stable stance. With the body held on a vertical rail the legs moved correctly, but released the robot did not balance or walk reliably. The Gazebo work was therefore set aside, and its lessons carried into the next attempt: the servo must be modelled as a torque-limited position loop, and the integrator must be stable at the stiffness that requires.

7.3.3 MUJOCO

The final simulation uses MuJoCo (Todorov et al., 2012), through its Python bindings. MuJoCo's position actuators implement exactly the required servo model, a proportional-derivative position loop whose output torque is clamped, and its implicit integrator is stable at the stiffness involved. The whole simulation, including the field, can be run without a display, so that every change could be tested by measuring falls, tilt, distance and servo load, and frames could be rendered to check its appearance, before it was shown interactively.

7.4 MUJOCO MODEL

The model is generated by a Python script from the dimensions in Table 4.1. Table 7.1 lists its main parameters. The body carries visual models of the sensors and the camera, and sites at the sensor positions: a front ultrasonic beam, a side beam towards the crop row, the gas sensor inlet and an onboard camera matching the ESP32-CAM's position.

Table 7.1: Parameters of the MuJoCo model

Parameter	**Value**
Time step and integrator	2 ms, implicit-fast
Total mass	1.35 kg (body 0.55 kg; each leg 0.20 kg)
Servo model	Position actuator, stiffness 15 N·m/rad, damping 0.15 N·m·s/rad
Servo torque limit	1.0 N·m (MG996R stall: 0.92 N·m at 4.8 V, 1.08 N·m at 6 V)
Joint damping, armature, friction	0.04 N·m·s/rad, 0.005 kg·m², 0.01 N·m
Foot contact	Rubber sphere, radius 13 mm, friction 1.2
Joint ranges	Abduction ±0.6 rad, hip ±1.7 rad, knee −2.75 to 0 rad

Parameter	Value
Sensors	IMU (orientation, gyro, accelerometer); two range finders
Controller rate	100 Hz (five physics steps per control step)

7.5 TROT GAIT

The gait is a diagonal trot: the front-left and rear-right legs move together, half a cycle apart from the front-right and rear-left legs. Each foot spends a fraction of the cycle, the duty factor, on the ground moving backward relative to the body, and the rest in the air moving forward. During stance the foot moves in a straight line; during swing it follows a raised-cosine path that lifts it 30 mm and lands it smoothly (Figure 7.2). The body is held 180 mm above the feet.

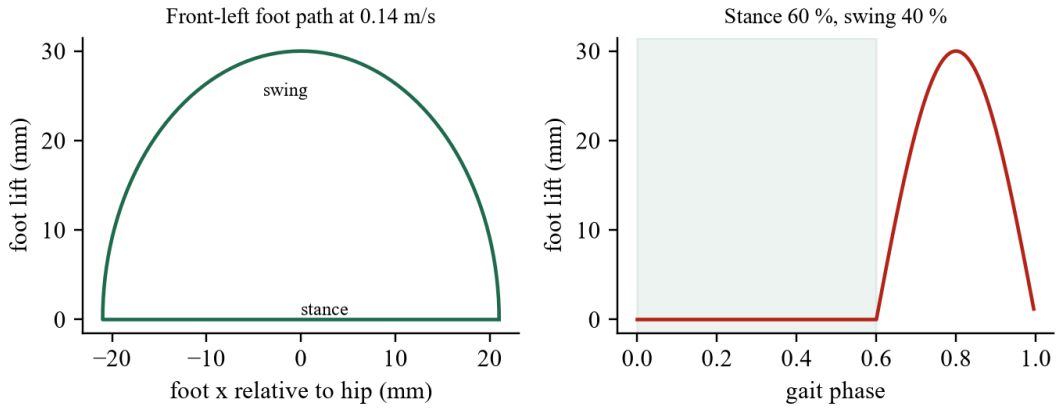


Figure 7.2: Path of the front-left foot over one gait cycle at 0.14 m/s (left) and its height against gait phase (right)

For a forward speed v and yaw rate ω, the stance stroke of the foot at position (x_i, y_i) relative to the body centre follows the ground velocity at that point:

$$\mathbf{v}_i = (v - \omega\, y_i,\ \ \omega\, x_i) \tag{7.5}$$

so the same gait both walks straight and turns, with feet on the outside of a turn taking longer strides. A levelling term reads the body's roll and pitch from the IMU and moves each foot vertically by half the displacement that tilt imposes at that foot, which damps rocking. When the robot stops, the stepping amplitude ramps to zero over 0.4 s so that all four feet come down together.

The gait parameters were chosen from a systematic sweep. Frequency (2, 2.5 and 3 Hz), duty factor (0.5 and 0.6), servo stiffness (15 and 30 N·m/rad) and levelling

gain (0 and 0.5) were combined into 24 settings, and each was run for eight seconds while recording falls, tilt and the fraction of time any servo was at its torque limit. The chosen setting, 2 Hz with a duty factor of 0.6, gives a short period in each cycle when all four feet are on the ground, which adds stability.

7.6 CENTRE-OF-MASS-CENTRED FOOTHOLDS

The first runs of the trot failed in an instructive way. In all 24 settings the robot tipped over within 0.8–3.0 s, even when stepping in place, and it always drifted backward. Recording the joint torques and body attitude showed the pitch increasing steadily nose-up and the rear knees saturating at their torque limit. The cause was the position of the centre of mass. All four knees point backward, so the leg mass hangs behind the hips, and the whole-body centre of mass lies 12 mm behind the geometric centre of the body.

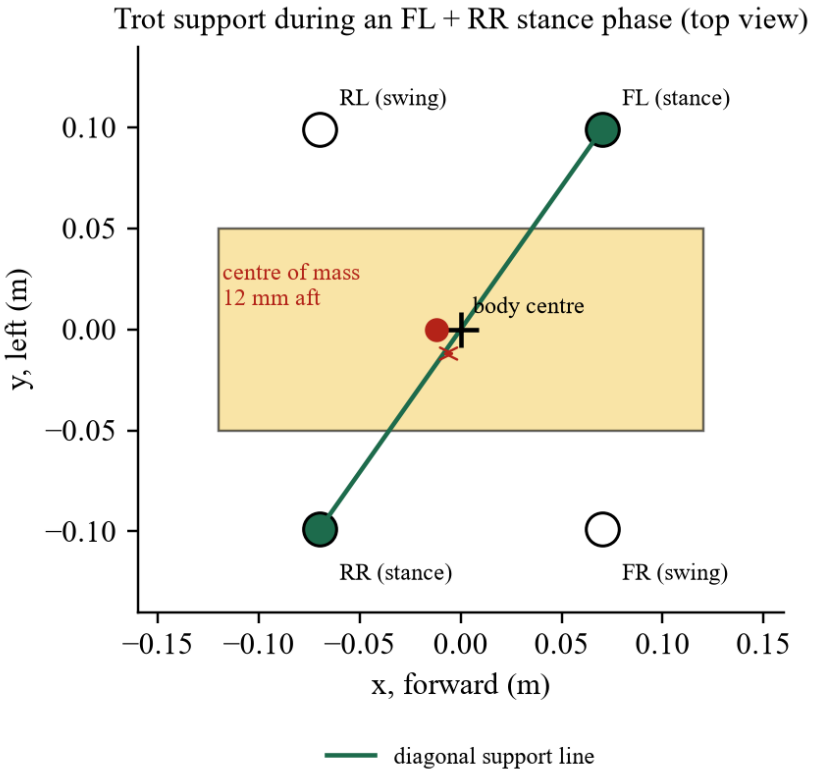


Figure 7.3: Top view during a trot stance phase: the diagonal support line passes through the body centre, in front of the centre of mass

In a trot the robot is supported for much of each cycle by only two diagonal feet, and the line between them passes through the centre of the footprint, which coincides with the body centre. With the centre of mass 12 mm behind that line (Figure 7.3),

gravity produces a moment about the support line in every stance phase, always tipping the body backward. The servos fight it until they saturate, and the error accumulates.

The fix is to place the footholds around the centre of mass rather than the body centre. The controller computes the whole-body centre of mass in the body frame at every control step from MuJoCo's subtree centre of mass, smooths it with a first-order filter, and shifts every foot target by that offset. With this change the same gait trotted in place with tilt below 0.7°, and walking and turning became stable. The cause lies in the geometry of the legs, not in the simulator, so the correction applies equally to the physical robot, and must be recomputed when payload is added or moved.

7.7 AUTONOMOUS SURVEY MISSION

7.7.1 SIMULATED FIELD

The simulated field contains four rows of tomato plants, 4 m long and 0.7 m apart, on raised soil beds with drip irrigation lines, surrounded by a hedge. Each plant has a stem, five to seven leaves and, for healthy plants, green and red fruit. A patch of plants around one point in the middle of the field has yellowed leaves, representing stressed or diseased plants. The plants are visual objects and targets for the range sensors, but the robot does not collide with them.

7.7.2 ENVIRONMENT AND SENSOR MODELS

The air in the field is described by three modelled fields. TVOC has a background of 16 ppb and a plume centred on the stressed patch, elongated along the rows:

$$\mathrm{TVOC}(x, y) = 16 + 640 \exp[-(\Delta x^2 + 2.2\Delta y^2) / (2 \cdot 0.42^2)] \ \text{ppb} \qquad (7.6)$$

where $(\Delta x, \Delta y)$ is the offset from the centre of the patch in metres. Dust is raised over the dry headland at one end of the field, and temperature varies gently across it. The simulated SGP30 follows the local TVOC through a first-order lag with a 2.5 s time constant, matching the slow response of real metal-oxide sensors, and eCO_2 is derived from TVOC; the simulated PPD42 averages over a longer time. Small random noise is added to every reading. These fields are models chosen to exercise the system, not measurements, and results based on them demonstrate the method rather than field performance.

7.7.3 MISSION LOGIC

The robot starts crouched at one end of the first gap between rows, stands up over two seconds, and then follows a serpentine route down each of the three gaps in turn. Heading is controlled by comparing the direction to the next waypoint with the yaw from the IMU. The yaw-rate command is 1.6 times the heading error, limited to ±0.55 rad/s, and the forward speed of 0.14 m/s is scaled by the cube of the cosine of the heading error, so that the robot slows smoothly when it needs to turn and turns on the spot when the error exceeds 0.6 rad.

The front ultrasonic sensor decides when a row ends. While the robot is heading down a gap, a reading below 0.55 m means it is facing the hedge at the end of the row, and the robot moves on to the next waypoint and turns into the next gap. The positional waypoints act only as a fallback. Every ten centimetres travelled, the TVOC reading is recorded with the robot's position, building a map of the air along its route. The mission ends with the robot standing still after the third pass.

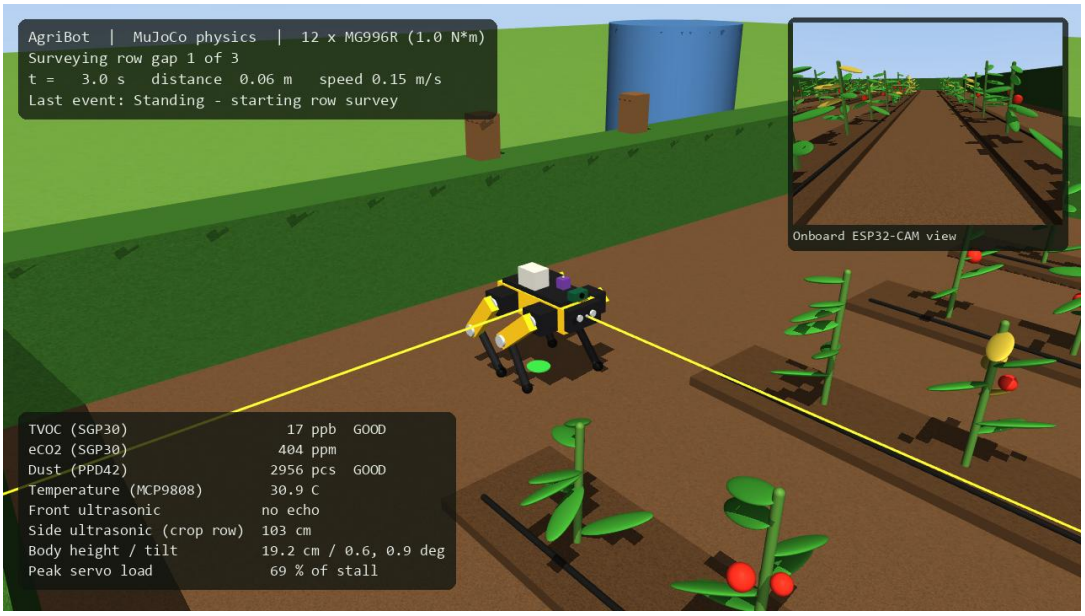


Figure 7.4: Start of the survey: the robot standing at the head of the first row gap, with the live sensor panels and the onboard camera view

7.7.4 INTERACTIVE VIEWER AND RECORDING

The simulation can be watched interactively in MuJoCo's viewer, with a camera that follows the robot, text panels showing the mission state and every simulated sensor reading, the onboard camera view inset in one corner, and the TVOC trail drawn on the ground in colours from green to red. It can also be run without a display to record a

video and produce the result plots of Chapter 8, and it can post its readings to the dashboard so that the same interface displays the simulated robot's data.

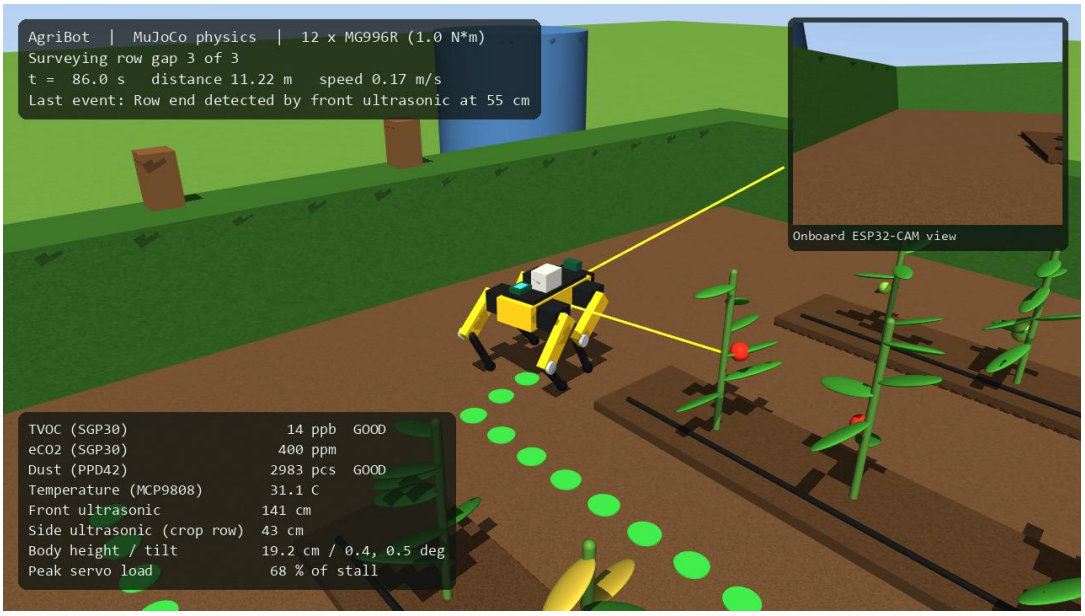


Figure 7.5: Turning in the headland between two passes; the trail behind the robot records the TVOC measured along its route

7.8 SUMMARY

The kinematics are exact and verified against the simulator. The simulation went through three generations; the third, in MuJoCo, models the servos with their real torque limit and supports a complete autonomous survey. Its most important result emerged before any walking succeeded: the robot's centre of mass sits behind its body centre, and the gait only works once footholds account for that.

Chapter 8
Results and Discussion

8.1 INTRODUCTION

This chapter reports the results in two groups. The first comes from the physical hardware: bench tests of each module, logged sensor sessions, video performance and classification. The second comes from the MuJoCo simulation: kinematic verification, gait stability, the speed limit imposed by the servos, and the autonomous survey. The chapter ends by comparing the outcome with the objectives and stating the limitations of the work.

8.2 BENCH VALIDATION OF THE HARDWARE

Each working module was exercised on the bench with a test that produced a known change in its input. Table 8.1 summarises the results.

Table 8.1: Bench tests of the physical sensing system

Test	Result	What it shows
SGP30, breath at a few centimetres	eCO_2 400 → 746 ppm and TVOC 0 → 210 ppb within 5 s; back to baseline once the air cleared	Gas sensing responds and recovers
MCP9808, finger contact	31.5 → 35.0 °C in about 10 s	Temperature tracks a real change
Sensor node Wi-Fi	−78 dBm with repeated authentication time-outs; −26 dBm after fitting the antenna	Diagnostics located the fault
Camera image quality (factory firmware)	640 × 480, correctly exposed, mean brightness 113 of 255	Camera and OV3660 sensor working
Camera video on the dashboard	About 0.5 → 16–20 frames per second after moving to MJPEG streaming	Live view usable for scanning
Camera registration	Appeared on the dashboard on a new address at every boot tested	No manual network setup needed
PPD42 disconnected	Reported as absent instead of a false count of 1.1 million	No false alarms from wiring faults
PPD42 connected, room air	0–5,065 particles per 0.01 ft^3, varying from one 30 s window to the next; about 2,000 on 18 October	Dust sensing live
Servos, robot on its back	All four legs move through commanded joint angles	Wiring and servo control working

Test	Result	What it shows
Servos, robot standing	Holds a stance and shifts its legs, but slowly and with little reserve	Torque margin is thin, as predicted

8.3 LOGGED SENSOR SESSIONS

Two sessions logged by the dashboard from the physical sensor node are shown here. The first, on 12 October 2026, lasted 28 minutes and recorded 321 readings (Figure 8.1). The air stayed in the good band throughout, with eCO_2 between 400 and 449 ppm and TVOC between 0 and 101 ppb. The temperature trace shows a sharp rise to 35.3 °C about 15 minutes in, during a finger-contact test, followed by a return to the 31.5 °C baseline over about two minutes, which reflects the thermal mass of the sensor board. The received signal strength stayed between −26 and −60 dBm, comfortably strong for the five-second telemetry rate, and no records were lost.

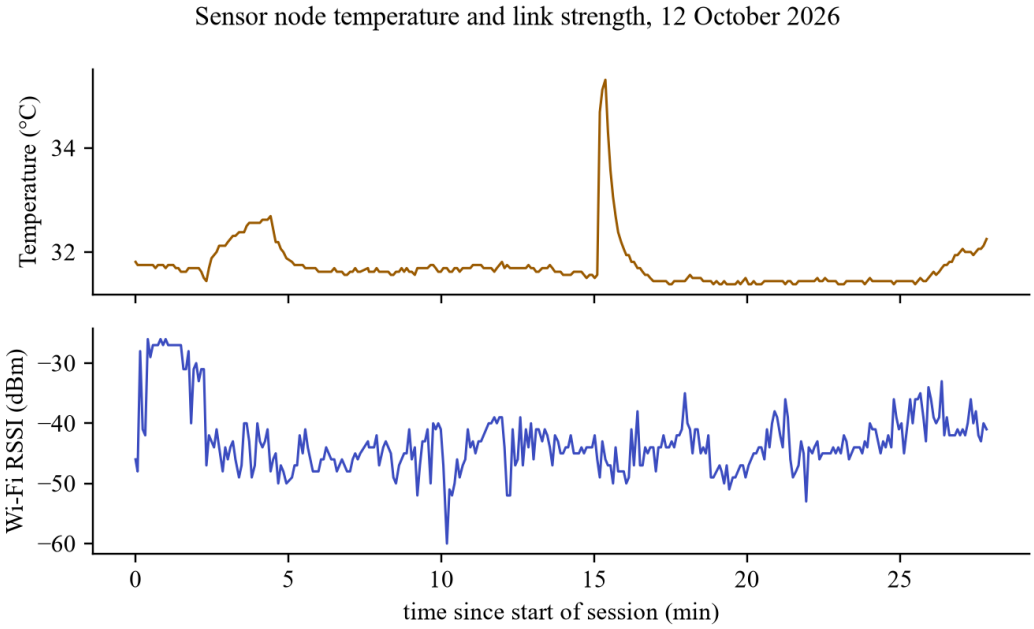


Figure 8.1: Temperature and Wi-Fi signal strength logged from the sensor node over 28 minutes on 12 October 2026

The second session, on 14 October 2026, lasted 17 minutes and recorded 199 readings (Figure 8.2) while the gas sensor was repeatedly exposed to VOC sources. TVOC reached 1,764 ppb and eCO_2 13,117 ppm; 80 readings fell in the high band, 48 in the moderate band and 71 in the good band. After each exposure both quantities fell back towards baseline within tens of seconds. Temperature drifted slowly upward by about one degree over the session. These sessions show the sensor node and dashboard

working continuously over Wi-Fi, logging without gaps and grading readings as designed.

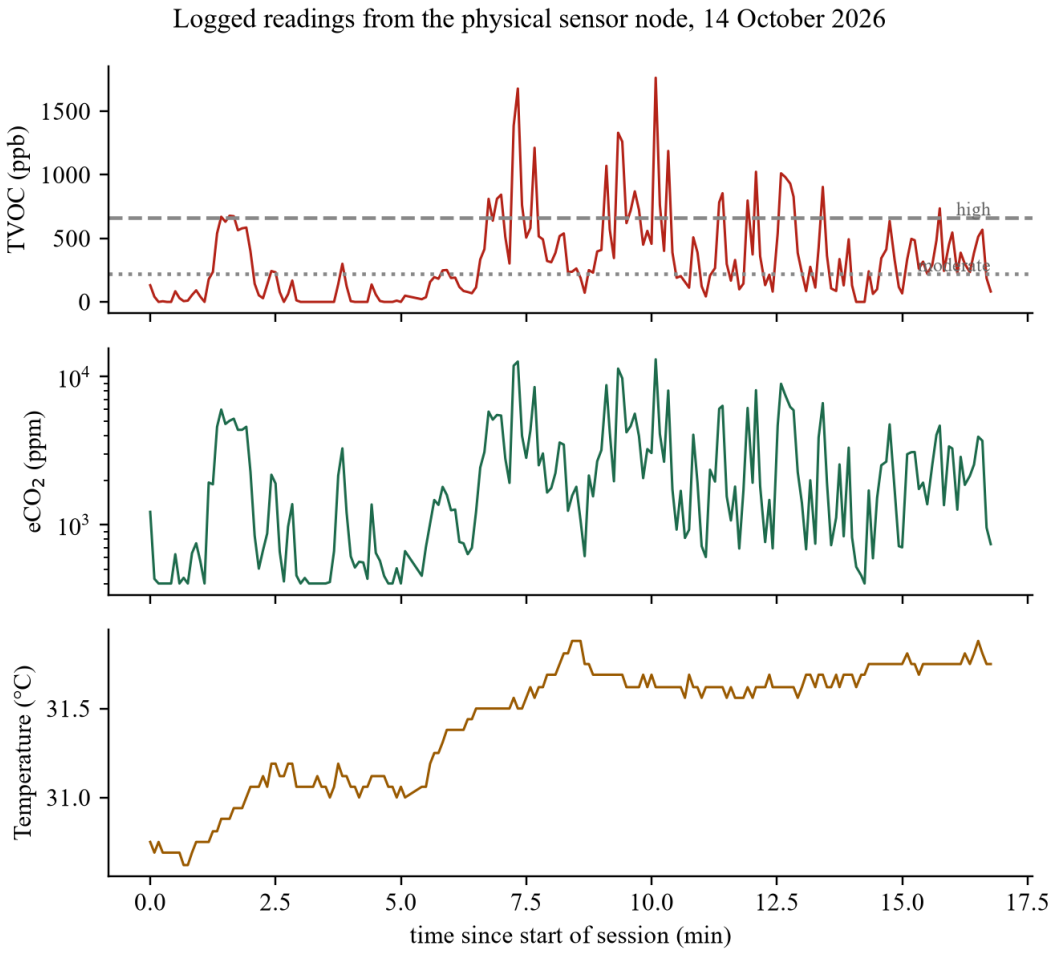


Figure 8.2: TVOC, eCO_2 (logarithmic scale) and temperature logged from the sensor node over 17 minutes on 14 October 2026; dashed lines mark the moderate and high TVOC thresholds

8.4 VIDEO STREAMING

With the original design, one still image fetched every second and a page that reloaded it every two seconds, the user saw about one new frame every two seconds. After the change to streaming described in Chapter 6, the dashboard measured 16–20 frames per second at 640 × 480, with frames of about 11–18 KB. Scanning a leaf during streaming worked without interrupting the video, because stills and the stream are served by separate servers on the camera.

8.5 LEAF-DISEASE CLASSIFICATION

Table 8.2 lists the scans logged by the dashboard on 14 October 2026, together with the first room scan from 12 October. Two results stand out. Pointed at the room, the classifier on its own returned "orange with citrus greening" at 82–90 % confidence; the vegetation gate measured no leaf coverage and rejected both frames. Of two live scans of leaves held to the camera, one was accepted and classified as cedar apple rust at 96.1 % with 60 % leaf coverage; the other contained only 18.6 % leaf coverage, passed the gate, but received a top probability of 36.4 % and was correctly reported as uncertain rather than given a label.

Table 8.2: Scans logged by the dashboard

Input	**Leaf coverage**	**Top probability**	**Reported result**
Room scene, camera (12 Oct)	—	89.7 %	Before the gate existed: "citrus greening"
Room scene, camera (14 Oct)	0.0 %	82.1 %	No leaf in view
12 PlantVillage images, upload	27–76 %	77.9–100 %	All accepted and classified
Leaf held to camera	18.6 %	36.4 %	Uncertain
Rust-affected leaf, camera	60.1 %	96.1 %	Apple, cedar apple rust
Hedge at about 1 m, camera (18 Oct)	11 %	75.5 %	No leaf in view
Potted strap-leaved plant, camera (18 Oct)	60 %	53.4 %	Corn (maize), healthy
Photograph of spotted leaves on a screen, camera (18 Oct)	26 %	80.5 %	Grape, black rot

Three further scans were taken outdoors with the camera on 18 October 2026 (Figure 8.3–Figure 8.5). They show the behaviour of the system on scenes that are not laboratory leaf photographs, and the cases where its answer should be treated with care.

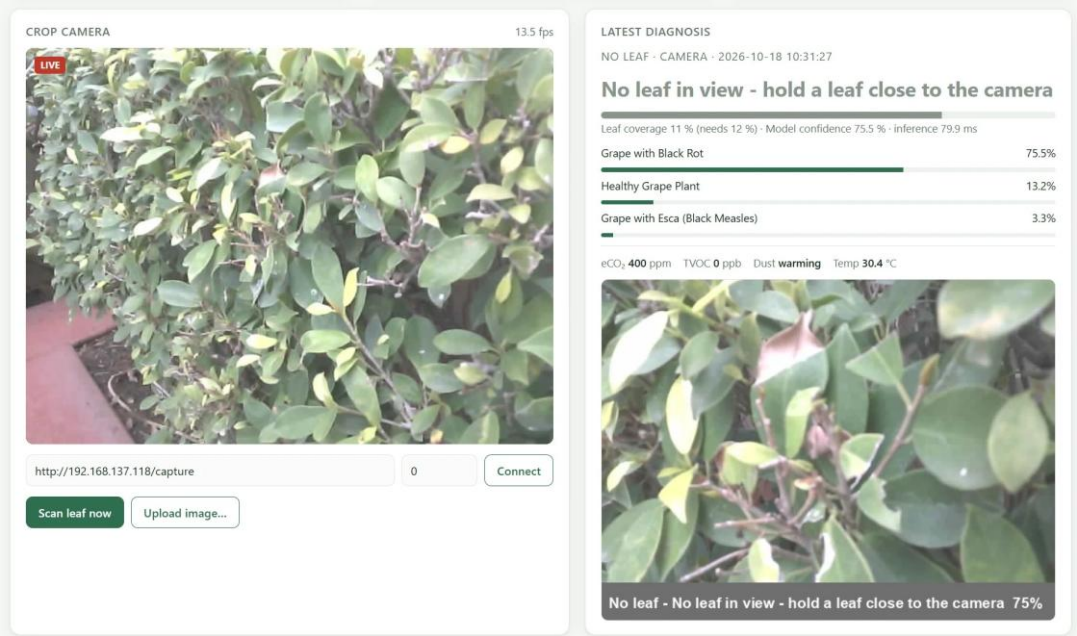


Figure 8.3: Scan of a hedge from about a metre away: leaf coverage 11 %, just below the 12 % threshold, so the classifier's 75.5 % grape black rot is withheld and the user is asked to bring a leaf closer

In Figure 8.3 the camera saw a whole hedge rather than a single leaf. The pale, washed-out colours of the ESP32-CAM image brought the saturation of many leaf pixels below the gate's limit, and the measured coverage of 11 % fell just short of the 12 % threshold. The classifier on its own would have reported grape black rot at 75.5 %; the gate withheld that label and asked for a leaf to be held closer. This is the intended outcome, since the model was trained on single leaves against plain backgrounds, but it also shows that the threshold sits close to the coverage of real foliage in this camera's images.

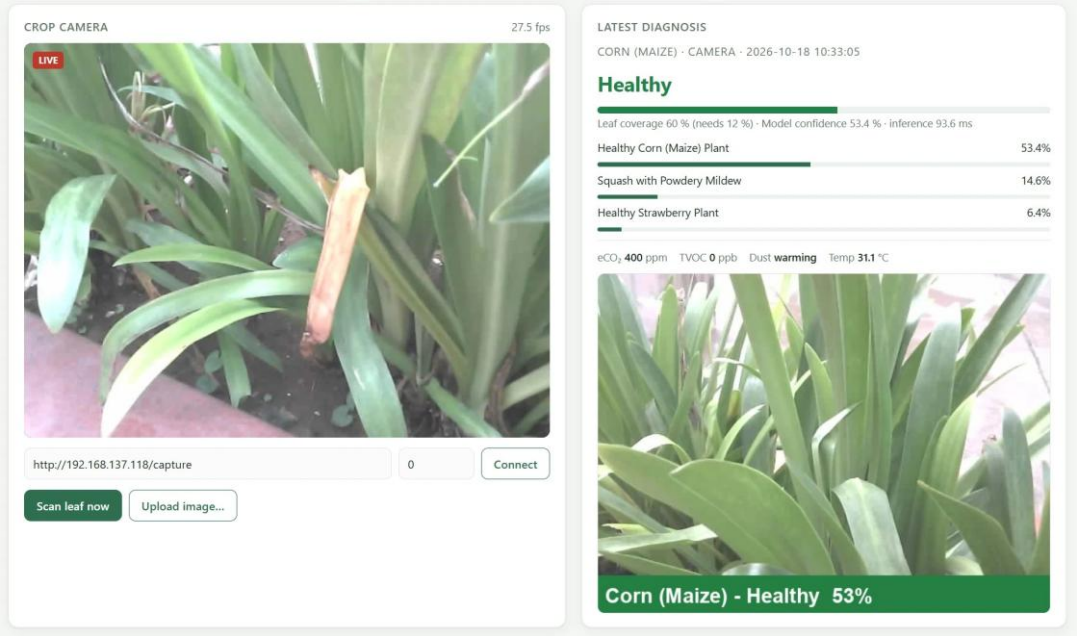


Figure 8.4: Scan of a potted plant with long strap-shaped leaves: leaf coverage 60 %, labelled healthy corn (maize) at 53.4 %, just above the 0.5 reporting threshold

In Figure 8.4 the frame was mostly leaf and passed the gate easily. The classifier chose healthy corn at 53.4 %, with squash powdery mildew second at 14.6 %. The species of the plant was not independently identified, so this result is not counted as right or wrong. It illustrates the closed-set limitation discussed in Chapter 2: any plant passes the colour gate, the model must choose one of its 14 crops, and a low top probability is the only warning that the input may not match any class well.

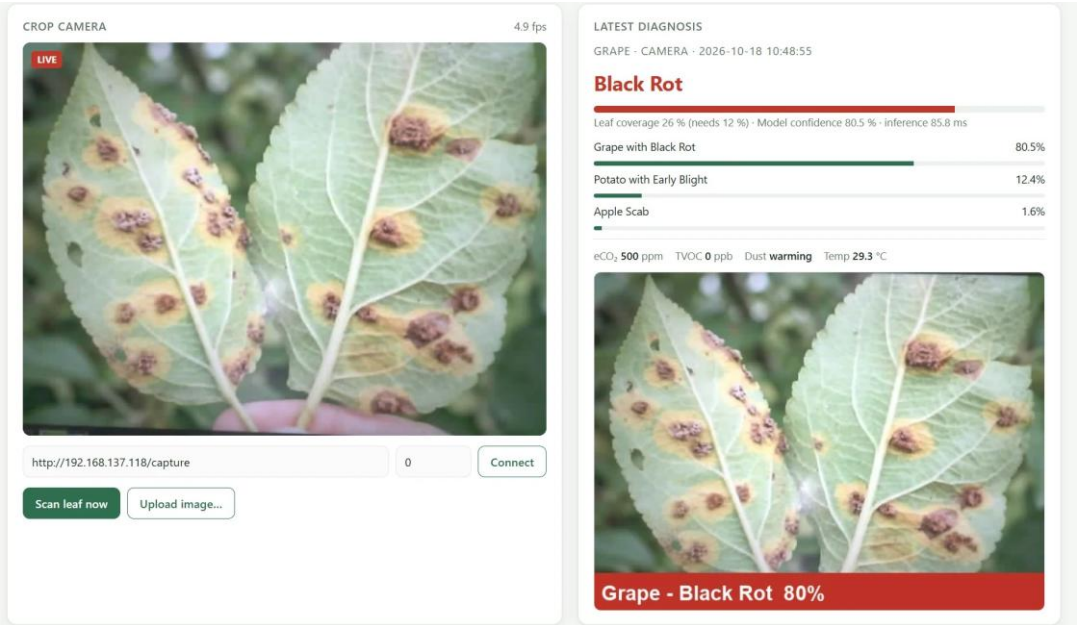


Figure 8.5: Scan of a photograph of spotted leaves displayed on a screen: leaf coverage 26 %, labelled grape black rot at 80.5 %

In Figure 8.5 a photograph of leaves with brown lesions and yellow halos was shown to the camera on a screen. The system flagged the leaf as diseased, with black rot at 80.5 % ahead of potato early blight at 12.4 %, and recorded the gas, dust and temperature readings at capture alongside the tagged image. The true condition of the leaves in the photograph was not verified. Taken together, the three scans show the gate and the uncertainty threshold doing their jobs, while confirming that the labels on non-PlantVillage scenes should be read as indications rather than diagnoses.

In an earlier spot check of ten PlantVillage images of known class, eight were classified correctly with confidence above 92 %, one was correct but fell below the 0.5 threshold and was reported as uncertain, and one tomato leaf-mould image was labelled late blight with 93 % confidence. On the laptop's processor, classification took 30–80 ms per image after the first call, which also loads the model. These checks show the pipeline and gate behaving as intended. They are not an accuracy measurement: the images are few, most come from the dataset the model was trained on, and accuracy on field photographs is expected to be lower (Mohanty et al., 2016).

8.6 ACTUATION TRIALS ON THE PHYSICAL ROBOT

Towards the end of the project the twelve MG996R servos of the printed robot were powered and driven for the first time. Two short trials were recorded on video. In the first (Figure 8.6) the robot lay on its back, so that the servos carried only the weight of the legs. All four legs moved through their commanded joint angles, which confirmed the servo wiring, the joint directions and the control signals.

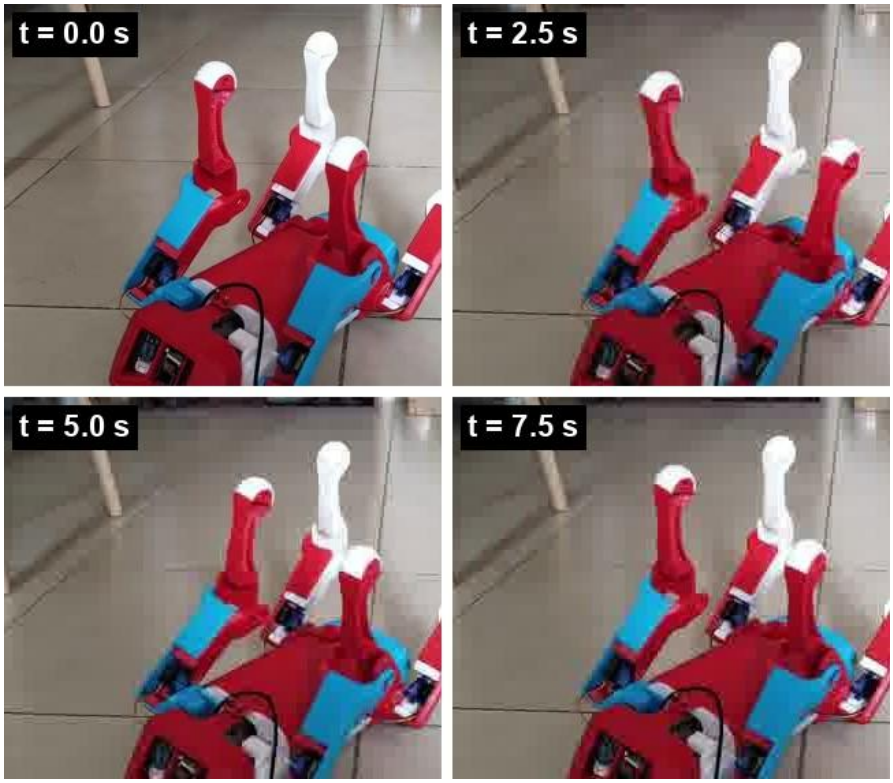


Figure 8.6: Leg actuation test with the robot on its back, frames from a 10 s video: the legs move through their commanded joint angles with no load on the feet

In the second trial (Figure 8.7) the robot stood on its feet and was commanded to move its legs while bearing its own weight. It held a stance and shifted its legs and body, but the movement was slow, of small amplitude and visibly strained, and the robot did not take a step. In short, the actuation chain works but is weak under load.

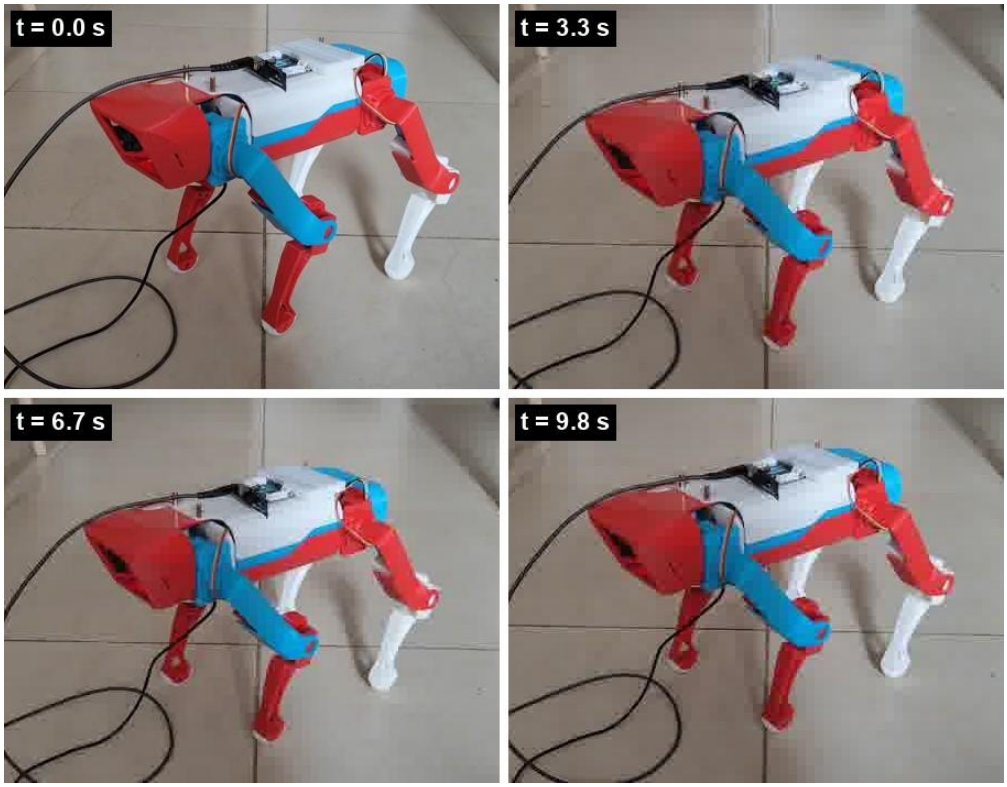


Figure 8.7: Standing test, frames from a 10 s video: the robot supports its own weight and shifts its legs, but slowly and with little reserve

This outcome is consistent with the analysis of Chapter 4 and the simulation. The static knee torque in a four-foot stance is 36–52 % of the rating of a genuine MG996R, and the share rises as a leg lifts and the remaining legs take its load; inexpensive copies of the servo often deliver well below the rated torque. Twelve servos moving at once can also draw several amperes, so a supply that sags under load reduces the available torque further. The likely remedies, in order of cost, are a dedicated 6 V supply rated for the peak current, the centre-of-mass-centred footholds found in simulation, a taller stance with straighter legs, which shortens the knee lever arm, and finally higher-torque servos. These causes have not yet been separated by measurement.

8.7 KINEMATIC VERIFICATION

The inverse kinematics of (7.1)–(7.4) were compared with MuJoCo's forward kinematics at 1,280 foot targets covering 8 forward positions, 5 lateral offsets and 8 heights for each of the four legs. The largest difference between target and achieved foot position was below 10^{-7} m. The kinematics therefore contribute no measurable error to the simulation results that follow.

8.8 GAIT STABILITY

Table 8.3 summarises the gait experiments. With footholds centred on the body, every one of the 24 settings fell within 0.8–3.0 s, in many cases with servos at their torque limit more than half the time. With footholds centred on the measured centre of mass, the selected setting trotted in place for the full test with tilt below 0.7°, walked straight at commanded speeds with yaw drift below 2°, and turned at 113–117 % of the commanded rate; the heading controller absorbs this overshoot.

Table 8.3: Gait experiments in simulation (2 Hz trot, duty factor 0.6 unless stated)

Experiment	Result
24 settings, footholds centred on the body	All fell within 0.8–3.0 s; consistent backward drift
Trot in place, CoM-centred footholds	No fall in 8 s; tilt ≤ 0.7°; a servo at its limit 8 % of the time
Walk 0.10 m/s for 12 s	1.15 m travelled; tilt ≤ 1.5°
Walk 0.16 m/s for 12 s	1.81 m travelled; tilt ≤ 2.5°; a servo at its limit 22 % of the time
Turn on the spot, 0.5 rad/s commanded	0.567 rad/s achieved (113 %)
Turn on the spot, 0.3 rad/s commanded	0.340 rad/s achieved (113 %)
Walk 0.10 m/s while turning 0.3 rad/s	0.350 rad/s achieved (117 %)

8.9 SPEED LIMIT IMPOSED BY THE SERVOS

To find how fast the robot can trot within the MG996R torque limit, it was commanded at ten speeds from 0.06 to 0.24 m/s for 15 s each (Table 8.4, Figure 8.8).

Table 8.4: Trot speed sweep with servo torque capped at 1.0 N·m

Commanded (m/s)	Achieved (m/s)	Time at torque limit	Outcome
0.06	0.053	8 %	Stable
0.08	0.075	8 %	Stable
0.10	0.096	8 %	Stable
0.12	0.115	8 %	Stable
0.14	0.134	15 %	Stable
0.16	0.151	22 %	Stable

Commanded (m/s)	Achieved (m/s)	Time at torque limit	Outcome
0.18	0.168	34 %	Stable
0.20	0.147	68 %	Tilt 12°
0.22	0.102	78 %	Fell
0.24	0.166	77 %	Tilt 16°

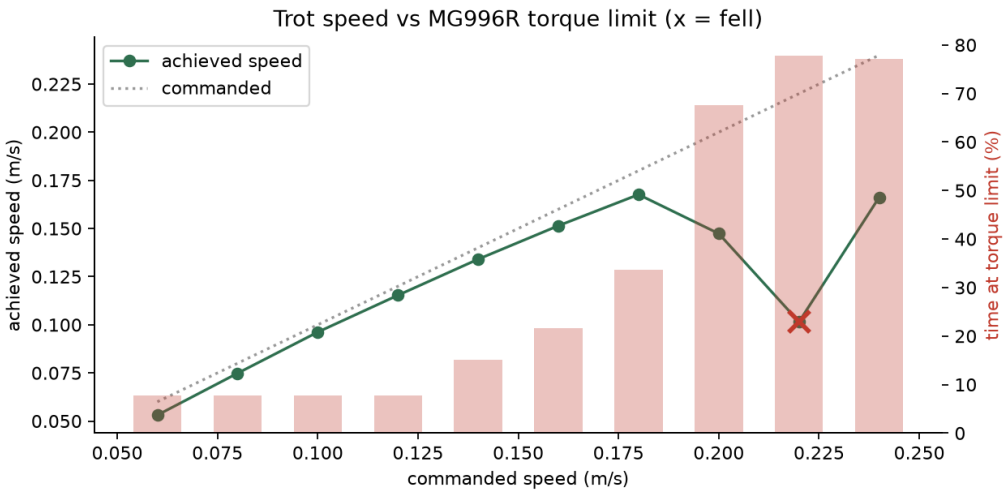


Figure 8.8: Achieved speed (line) and share of time a servo was at its torque limit (bars) against commanded speed; the cross marks a fall

From 0.08 to 0.18 m/s the robot tracked the command to within 7 %, with the share of time at the torque limit rising steadily from 8 % to 34 %. At 0.20 m/s the servos were at their limit two-thirds of the time, the achieved speed fell below the command and tilt grew to 12°. At 0.22 m/s the robot fell. The run at 0.24 m/s stayed upright but reached only 0.166 m/s with tilt of 16° and 77 % saturation, so it is not usable. Within a 1.0 N·m torque limit, this robot therefore has a practical speed ceiling of about 0.18 m/s; the survey runs at 0.14 m/s to keep a margin.

8.10 AUTONOMOUS FIELD SURVEY

The complete survey ran without a fall (Table 8.5). The robot stood up, walked all three gaps, detected each row end with the front ultrasonic sensor at 55 cm from the hedge, turned in the headland, and stopped after the third pass. Figure 8.9 shows it crossing the stressed patch, with the TVOC trail turning red.

Table 8.5: Results of the simulated autonomous survey

Metric	Value
Outcome	All 3 row gaps surveyed; no falls

Metric	Value
Distance and duration	16.7 m in 129 s
Mean speed	0.131 m/s (commanded 0.14 m/s)
Maximum body tilt	2.7°
Median of peak servo load per control step	62.5 % of stall
Time any servo at its torque limit	1.3 %
Row ends detected by front ultrasonic	3 of 3
TVOC peak and its position	535 ppb at x = 0.60 m (source at x = 0.80 m)
Dust peak (headland)	11,199 particles per 0.01 ft³



Figure 8.9: Simulated robot crossing the stressed patch; the TVOC reading is in the moderate band and the trail behind it is red

Figure 8.10 shows the route from above, coloured by the TVOC the robot measured, over the modelled field. The measured peak lies 0.2 m from the true centre of the plume. The robot crossed the plume moving in the negative *x* direction, and the gas sensor's 2.5 s response time shifted the peak along the path by roughly the distance covered in that time. The robot could therefore locate a gas source to within a fraction of a metre, but systematically behind its true position; a slower pass, a pause at each waypoint or compensating for the known sensor lag would reduce this error.

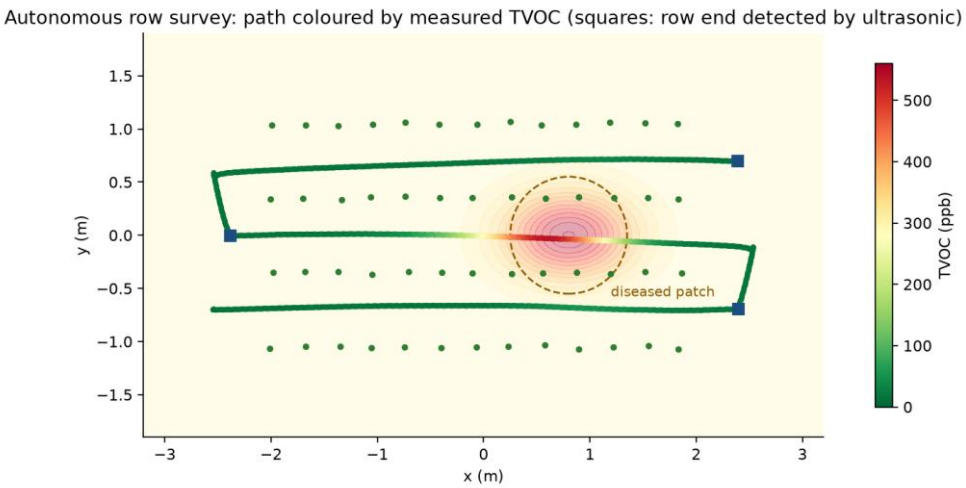


Figure 8.10: Survey route coloured by measured TVOC over the modelled VOC field; squares mark row ends detected by the front ultrasonic sensor

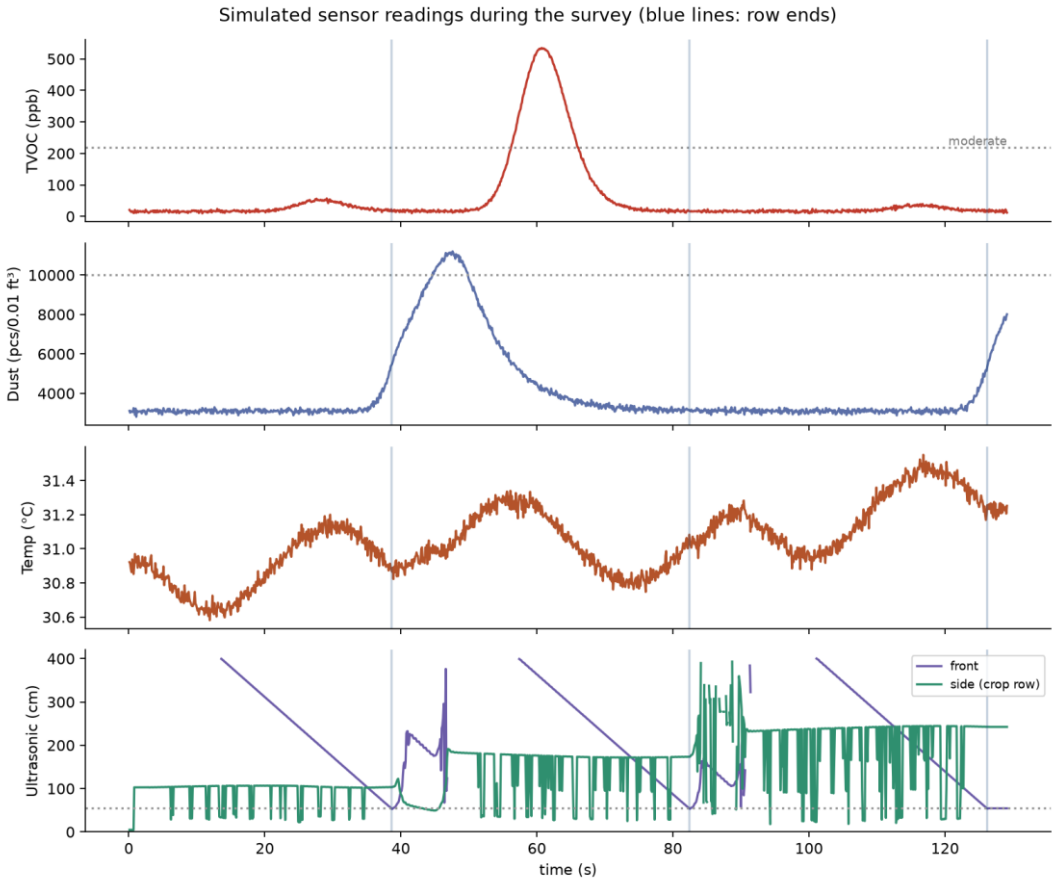


Figure 8.11: Simulated sensor readings during the survey; vertical lines mark row ends

Figure 8.11 shows the readings over time. TVOC rises only during the middle pass, which runs through the stressed patch, and dust peaks each time the robot turns in the dry headland at the far end. The front range falls steadily along each pass as the hedge approaches and triggers the turn at 55 cm. Figure 8.12 shows the locomotion

quality: the peak servo load stays well below the limit most of the time, with brief excursions during turns, and tilt stays within a few degrees throughout.

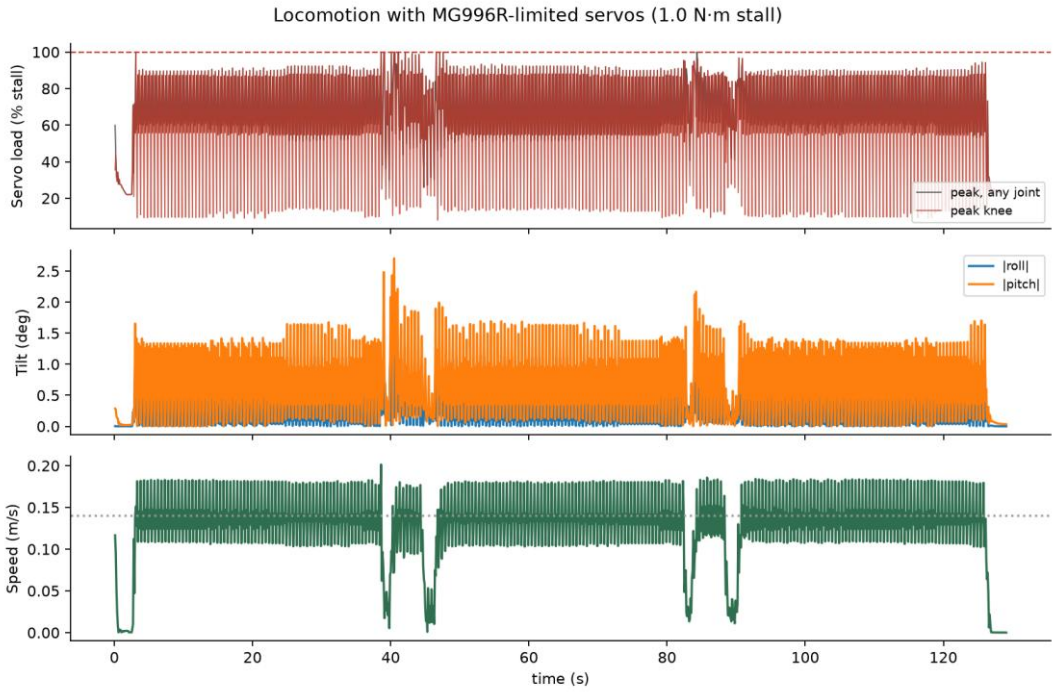


Figure 8.12: Servo load, body tilt and speed during the survey

8.11 COMPARISON WITH THE OBJECTIVES

Table 8.6: Objectives and their status

Objective	**Status**	**Evidence**
Fabricate and adapt the chassis; measure its geometry	Achieved	35 parts printed; four custom parts; measured geometry (Chapter 4)
Wireless sensing payload, robust to faults	Partly achieved	Gas, dust and temperature live; ultrasonic firmware complete, wiring pending
Camera streaming and classification with non-leaf rejection	Achieved	16–20 fps; gate rejects room scenes; live diagnosis (Section 8.5)
Live dashboard with logging and tagging	Achieved	Logged sessions; tagged captures (Sections 8.3, 8.5)
Verified kinematics; gait feasibility in simulation	Achieved	IK error < 10^{-7} m; complete survey without a fall
Walking on the physical robot	Partly achieved	Legs actuate and the robot stands, but weakly; no steps yet (Section 8.6)

Compared with the expectations stated at Review III, inference moved from the robot to a laptop, which made the system cheaper and safer at the cost of depending on

the Wi-Fi link. The Review III expectation of above 90 % classification accuracy was a figure from the literature on curated data; this project did not measure field accuracy, and its classification results should be read as a functional demonstration. The torque analysis of Review III was confirmed and extended: the simulation reproduced the importance of the knee and quantified the speed at which the servos run out of torque.

8.12 LIMITATIONS

- The gas, dust and temperature fields in the simulation are modelled, not measured, so the simulated survey demonstrates the method rather than field performance.
- The classifier was trained on laboratory photographs of 14 crops and has not been evaluated on field images. The vegetation gate cannot tell a trained crop from an untrained one, or a photograph of a leaf from a plant.
- The SGP30's eCO_2 is estimated from VOCs rather than measured, and the PPD42 is an uncalibrated particle counter; both give relative indications, not regulatory values.
- Walking has been shown only in simulation. On the physical robot the legs actuate and the robot stands, but the servos have too little reserve to step. The simulated servo is an ideal torque-limited position loop, while real MG996R servos add a deadband, backlash, unit-to-unit variation and sensitivity to supply voltage.
- The ultrasonic sensors are simulated as exact rays; real HC-SR04 sensors have a beam about 15° wide and can miss thin stems.
- At the time of writing the ultrasonic sensors were supported by the firmware but not yet wired on the physical node.

8.13 DISCUSSION

Three lessons from the results are worth drawing out. First, the most valuable output of the simulation was a fact about the robot rather than a gait: the centre of mass sits behind the body centre because of the leg geometry, and no tuning of gains or frequencies could compensate for footholds placed in the wrong place. This applies directly to the physical build. Second, robustness features that seem minor, such as detecting an absent sensor, reporting why Wi-Fi failed and letting the camera announce

itself, each removed a failure that occurred in practice during this project. Third, the vegetation gate shows that a small piece of domain knowledge can fix the most visible failure of a general model. It is not a substitute for open-set recognition, but it stops the system from presenting a confident disease report for a picture of a wall.

8.14 SUMMARY

On the hardware side, the gas, dust, temperature, video and classification modules work end to end and have been logged over extended sessions. In simulation, the robot walks, turns and completes an autonomous survey within realistic servo limits, once its footholds account for the measured centre of mass, and its speed is limited to about 0.18 m/s by servo torque. On the physical robot the legs actuate and the robot stands, but the servos are too weak under load to step. Walking on the physical robot and wiring of the ultrasonic sensors are the main outstanding items.

Chapter 9
Conclusion and Future Work

9.1 CONCLUSION

This project set out to build a low-cost quadruped platform for in-field crop and air-quality monitoring. A twelve-degree-of-freedom chassis was printed from an adapted open design, its geometry measured, and four parts designed to carry the payload. After analysis of supply, heat, torque margin and the risk to an onboard computer in a fall, the design was revised so that two inexpensive Wi-Fi microcontroller nodes carry the sensors and a laptop carries the computation.

The sensing payload works end to end. The sensor node measures gas, dust and temperature, shows readings on its own screen, sounds alerts and reports every five seconds; its firmware for range sensing is complete. The camera streams live video at 16–20 frames per second and registers itself with the dashboard. The dashboard logs every reading, shows live charts and video, and classifies leaf disease, with a vegetation gate that rejects frames without a leaf: the classifier alone had labelled a room scene as a diseased orange leaf with up to 90 % confidence.

Locomotion was established in physics simulation with realistic servo limits. The leg kinematics were verified against the physics engine. The simulation revealed that the robot's centre of mass sits 12 mm behind its body centre, and that the trot is stable only when footholds are placed around the measured centre of mass. With that correction the simulated robot completed an autonomous survey of three crop rows without a fall, detected every row end with its ultrasonic sensor and located a simulated gas source to within 0.2 m. MG996R servos limit its trot to about 0.18 m/s.

On the physical robot, all twelve servos were driven and the robot supported its own weight, but it moved slowly and with little reserve and did not take a step, as the torque analysis had warned. The work has therefore not yet shown walking on the physical robot, and its environmental results in simulation rest on modelled fields. Within those limits, it shows that a printed quadruped and commodity electronics can form a credible monitoring platform, and it identifies the design factors, centre of mass, servo torque and classifier guarding, that decide whether such a platform works.

9.2 FUTURE WORK

1. **Walking on hardware.** Strengthen the actuation that already moves the legs: give the servos a dedicated 6 V supply rated for their peak current, measure voltage and torque under load to separate supply sag from servo weakness, and adopt the centre-of-mass-centred footholds and a taller stance. Then progress from standing and sitting to stepping in place with the simulated gait, and to slow walking below the measured speed ceiling, moving to higher-torque servos if the margin is still too thin.
2. **Complete the sensor set.** Wire both ultrasonic sensors through their dividers, add a soil-moisture probe on the reserved analogue pin, and calibrate the gas and dust readings against reference instruments.
3. **Act on measurements.** Use the Grove relay to switch a small irrigation pump when soil moisture is low, or to raise an alarm when gas readings are high, with a control on the dashboard.
4. **Field-ready classification.** Collect leaf images with the robot's own camera in a real field, fine-tune the classifier on them, and replace the colour gate with a learned open-set detector.
5. **Gas-source localisation.** Compensate for the gas sensor's response lag, either by pausing at waypoints or by deconvolving the readings with its known time constant, and record positions with odometry or GPS to build field maps.
6. **Untethered operation.** Add a battery with separate regulated supplies for logic and servos, and move to higher-torque servos if faster walking is required.

REFERENCES

[1] Bechar, A., & Vigneault, C. (2016). Agricultural robots for field operations: Concepts and components. *Biosystems Engineering, 149*, 94–111. https://doi.org/10.1016/j.biosystemseng.2016.06.014

[2] Bledt, G., Powell, M. J., Katz, B., Di Carlo, J., Wensing, P. M., & Kim, S. (2018). MIT Cheetah 3: Design and control of a robust, dynamic quadruped robot. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 2245–2252). IEEE. https://doi.org/10.1109/IROS.2018.8593885

[3] Elijah, O., Rahman, T. A., Orikumhi, I., Leow, C. Y., & Hindia, M. N. (2018). An overview of Internet of Things (IoT) and data analytics in agriculture: Benefits and challenges. *IEEE Internet of Things Journal, 5*(5), 3758–3773.

[4] Espressif Systems. (n.d.). *ESP32-S3 series datasheet* [Datasheet].

[5] Ferentinos, K. P. (2018). Deep learning models for plant disease detection and diagnosis. *Computers and Electronics in Agriculture, 145*, 311–318. https://doi.org/10.1016/j.compag.2018.01.009

[6] Fountas, S., Mylonas, N., Malounas, I., Rodias, E., Hellmann Santos, C., & Pekkeriet, E. (2020). Agricultural robotics for field operations. *Sensors, 20*(9), 2672. https://doi.org/10.3390/s20092672

[7] Grinberg, M. (2018). *Flask web development: Developing web applications with Python* (2nd ed.). O'Reilly Media.

[8] Hendrycks, D., & Gimpel, K. (2017). A baseline for detecting misclassified and out-of-distribution examples in neural networks. In *International Conference on Learning Representations (ICLR)*.

[9] Hughes, D. P., & Salathé, M. (2015). *An open access repository of images on plant health to enable the development of mobile disease diagnostics* (arXiv:1511.08060). arXiv.

[10] Hutter, M., Gehring, C., Jud, D., Lauber, A., Bellicoso, C. D., Tsounis, V., Hwangbo, J., Bodie, K., Fankhauser, P., Bloesch, M., Diethelm, R., Bachmann, S., Melzer, A., & Hoepflinger, M. (2016). ANYmal – a highly mobile and dynamic quadrupedal robot. In *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE. https://doi.org/10.1109/IROS.2016.7758092

[11] Hwangbo, J., Lee, J., Dosovitskiy, A., Bellicoso, D., Tsounis, V., Koltun, V., & Hutter, M. (2019). Learning agile and dynamic motor skills for legged robots. *Science Robotics, 4*(26), eaau5872. https://doi.org/10.1126/scirobotics.aau5872

[12] Kamilaris, A., & Prenafeta-Boldú, F. X. (2018). Deep learning in agriculture: A survey. *Computers and Electronics in Agriculture, 147*, 70–90. https://doi.org/10.1016/j.compag.2018.02.016

[13] Katz, B., Di Carlo, J., & Kim, S. (2019). Mini Cheetah: A platform for pushing the limits of dynamic quadruped control. In *2019 International Conference on Robotics and Automation (ICRA)*. IEEE. https://ieeexplore.ieee.org/document/8793865

[14] Kau, N., Schultz, A., Ferrante, N., & Slade, P. (2019). Stanford Doggo: An open-source, quasi-direct-drive quadruped. In *2019 International Conference on Robotics and Automation (ICRA)*. IEEE. https://doi.org/10.1109/ICRA.2019.8794436

[15] KDY0523. (n.d.). *SpotMicro – robot dog* [3D model]. Thingiverse. Retrieved October 2026, from https://www.thingiverse.com/thing:3445283

[16] Koenig, N., & Howard, A. (2004). Design and use paradigms for Gazebo, an open-source multi-robot simulator. In *2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (Vol. 3, pp. 2149–2154). IEEE.

[17] Liakos, K. G., Busato, P., Moshou, D., Pearson, S., & Bochtis, D. (2018). Machine learning in agriculture: A review. *Sensors, 18*(8), 2674. https://doi.org/10.3390/s18082674

[18] Macenski, S., Foote, T., Gerkey, B., Lalancette, C., & Woodall, W. (2022). Robot Operating System 2: Design, architecture, and uses in the wild. *Science Robotics, 7*, eabm6074. https://doi.org/10.1126/scirobotics.abm6074

[19] Martinelli, F., Scalenghe, R., Davino, S., et al. (2015). Advanced methods of plant disease detection. A review. *Agronomy for Sustainable Development, 35*(1), 1–25. https://doi.org/10.1007/s13593-014-0246-1

[20] Microchip Technology Inc. (n.d.). *MCP9808: ±0.5 °C maximum accuracy digital temperature sensor* [Datasheet].

[21] Mohanty, S. P., Hughes, D. P., & Salathé, M. (2016). Using deep learning for image-based plant disease detection. *Frontiers in Plant Science, 7*, 1419. https://doi.org/10.3389/fpls.2016.01419

[22] Ray, P. P. (2017). Internet of things for smart agriculture: Technologies, practices and future direction. *Journal of Ambient Intelligence and Smart Environments, 9*(4), 395–420. https://doi.org/10.3233/AIS-170440

[23] Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (pp. 4510–4520).

[24] Savary, S., Willocquet, L., Pethybridge, S. J., Esker, P., McRoberts, N., & Nelson, A. (2019). The global burden of pathogens and pests on major food crops. *Nature Ecology & Evolution, 3*(3), 430–439. https://doi.org/10.1038/s41559-018-0793-y

[25] Scheirer, W. J., Rocha, A. de R., Sapkota, A., & Boult, T. E. (2013). Toward open set recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 35*(7), 1757–1772. https://doi.org/10.1109/TPAMI.2012.256

[26] Sensirion AG. (n.d.). *SGP30: Indoor air quality sensor for TVOC and CO_2eq measurements* [Datasheet].

[27] Shinyei Technology Co. (n.d.). *Particle sensor unit PPD42NS* [Specification sheet].

[28] Todorov, E., Erez, T., & Tassa, Y. (2012). MuJoCo: A physics engine for model-based control. In *2012 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)* (pp. 5026–5033). IEEE. https://doi.org/10.1109/IROS.2012.6386109

[29] Too, E. C., Yujian, L., Njuki, S., & Yingchun, L. (2019). A comparative study of fine-tuning deep learning models for plant disease identification. *Computers and Electronics in Agriculture, 161*, 272–279.

[30] Tower Pro. (n.d.). *MG996R high-torque metal gear servo* [Datasheet].

[31] Tzounis, A., Katsoulas, N., Bartzanas, T., & Kittas, C. (2017). Internet of Things in agriculture, recent advances and future challenges. *Biosystems Engineering, 164*, 31–48. https://doi.org/10.1016/j.biosystemseng.2017.09.007

[32] Woebbecke, D. M., Meyer, G. E., Von Bargen, K., & Mortensen, D. A. (1995). Color indices for weed identification under various soil, residue, and lighting conditions. *Transactions of the ASAE, 38*(1), 259–269.

Appendix 1
Wiring and Pin Map

This appendix reproduces the wiring document used to assemble the sensor node. Figure A1.1 shows the breadboard layout of the three voltage dividers, each made of three 1 kΩ resistors, and Table A1.1 lists every connection.

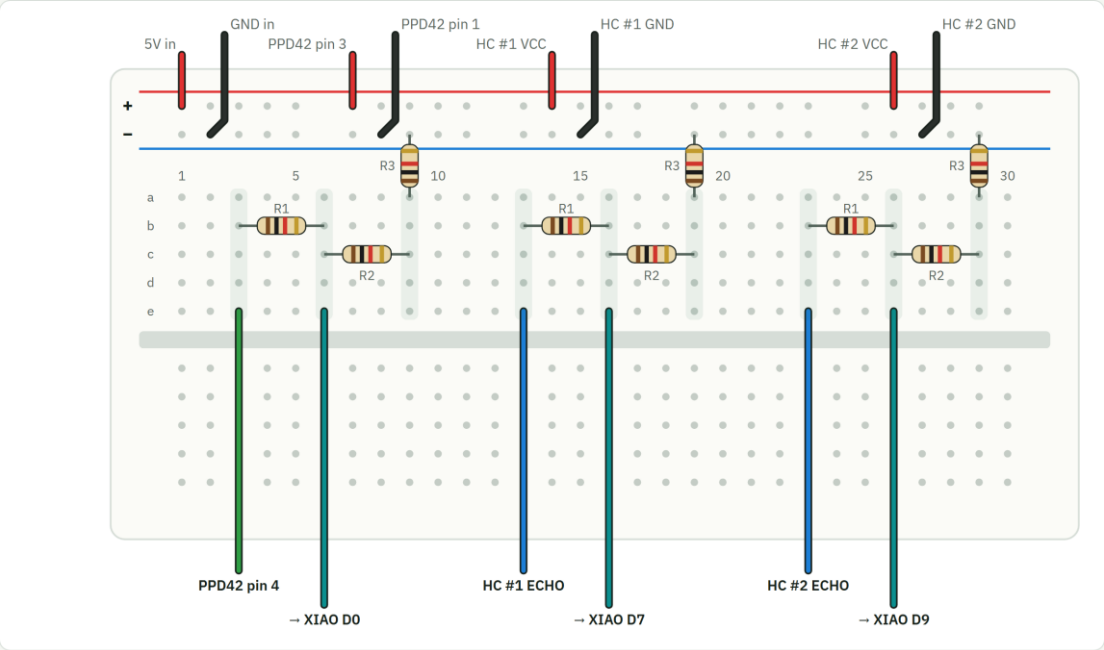


Figure A1.1: Breadboard layout: power rails fed from the expansion board, and dividers A, B and C for the PPD42 and the two ultrasonic echo signals

Table A1.1: Wire-by-wire connection list

#	From	To	Notes
1	SGP30 SCL	Grove IIC 1, SCL (D5)	Grove yellow
2	SGP30 SDA	Grove IIC 1, SDA (D4)	Grove white
3	SGP30 VCC	Grove IIC 1, 3V3	Grove red carries 3.3 V
4	SGP30 GND	Grove IIC 1, GND	Grove black
5	MCP9808	Grove IIC 2	Grove cable as supplied
6	Side header 5V	Breadboard + rail	
7	Side header GND	Breadboard − rail	
8	PPD42 pin 3 (5 V)	+ rail	
9	PPD42 pin 1 (GND)	− rail	
10	PPD42 pin 4 (P1)	Divider A input	

#	From	To	Notes
11	Divider A tap	XIAO D0	Between R1 and R2
12	HC-SR04 #1 VCC / GND	+ rail / − rail	
13	HC-SR04 #1 TRIG	XIAO D6 (TX6)	No divider needed
14	HC-SR04 #1 ECHO	Divider B input	
15	Divider B tap	XIAO D7 (RX7)	Between R1 and R2
16	HC-SR04 #2 VCC / GND	+ rail / − rail	
17	HC-SR04 #2 TRIG	XIAO D8	No divider needed
18	HC-SR04 #2 ECHO	Divider C input	
19	Divider C tap	XIAO D9	Between R1 and R2
20	XIAO USB-C	Laptop or power bank	Data-capable cable
21	ESP32-CAM	ESP32-CAM-MB adapter	5V to 5V, GND to GND
22	MB micro-USB	Power bank	Short cable avoids brownouts

Before the first power-up, the + and − rails should be checked for continuity with a multimeter; they must not be connected. The PPD42 must be mounted upright with its connector at the bottom, and the microSD slot must be left empty because its lines are shared with the downward ultrasonic sensor. The 3S battery must never be connected to the expansion board's battery socket, which accepts only a single 3.7 V cell.

Appendix 2
Telemetry Format and Dashboard Interface

The sensor node sends the following JSON record to the dashboard every five seconds. Unavailable readings are sent as null.

Example telemetry record from the sensor node

```
{
  "node": "xiao-s3", "uptime_s": 1834, "rssi": -41,
  "sgp30_ok": true, "eco2_ppm": 421, "tvoc_ppb": 23, "air_level":
"good",
  "dust_present": false, "dust_valid": false,
  "dust_ratio_pct": 0.000, "dust_pcs_per_0_01cf": 0.0, "dust_level":
"absent",
  "mcp9808_ok": true, "temp_c": 31.62,
  "front_cm": null, "obstacle": false, "down_cm": null, "canopy_cm":
null,
  "muted": false
}
```

Table A2.1: Dashboard HTTP interface

Method	Path	Purpose
POST	/api/sensors	Receive a telemetry record; log it to the daily CSV file
GET	/api/state	Latest reading, ten-minute history, node, camera and classifier status, recent detections
POST	/api/camera	Set the camera's still and stream URLs (sent by the camera at boot)
GET	/api/camera/stream	Relayed MJPEG video for browsers
GET	/api/camera/frame.jpg	Latest single frame
POST	/api/scan	Run the vegetation gate and classifier on the current frame
POST	/api/classify	Run the vegetation gate and classifier on an uploaded image
GET	/captures/<file>	Saved, tagged images
GET	/api/export.csv	Download the current day's sensor log

Table A2.2: Camera node HTTP interface

Method	Path	Purpose
GET	:81/stream	Continuous MJPEG video
GET	:80/capture	One fresh JPEG; ?flash=1 lights the onboard LED
GET	:80/	Status: address, signal strength, uptime, frame counts

Appendix 3
Key Source Code Listings

The listings below are taken unchanged from the project repository.

Listing A3.1: Interrupt-driven PPD42 timing (sensor node firmware, C++)

```
portMUX_TYPE dustMux = portMUX_INITIALIZER_UNLOCKED;
volatile uint32_t dustLowAccumUs = 0;
volatile uint32_t dustLowStartUs = 0;
volatile bool dustIsLow = false;
volatile uint32_t dustEdges = 0;

void IRAM_ATTR onDustEdge() {
  uint32_t now = micros();
  portENTER_CRITICAL_ISR(&dustMux);
  dustEdges = dustEdges + 1;
  if (digitalRead(PIN_DUST) == LOW) {
    if (!dustIsLow) { dustIsLow = true; dustLowStartUs = now; }
  } else if (dustIsLow) {
    dustLowAccumUs += now - dustLowStartUs;
    dustIsLow = false;
  }
  portEXIT_CRITICAL_ISR(&dustMux);
}

// Returns LOW time in the window just ended and starts a new one. A
pulse still
// in progress is split at the window boundary so no time is lost or
double counted.
uint32_t takeDustLowUs(uint32_t &edges) {
  uint32_t now = micros();
  portENTER_CRITICAL(&dustMux);
  uint32_t lowUs = dustLowAccumUs;
  if (dustIsLow) { lowUs += now - dustLowStartUs; dustLowStartUs = now;
}
  dustLowAccumUs = 0;
  edges = dustEdges;
  dustEdges = 0;
  portEXIT_CRITICAL(&dustMux);
  return lowUs;
}
```

Listing A3.2: Dust window evaluation with absent-sensor detection (C++)

```
void closeDustWindow(uint32_t windowMs) {
  uint32_t edges;
  uint32_t lowUs = takeDustLowUs(edges);
  // A working PPD42 idles HIGH and pulses LOW. A line held LOW for a
whole
  // window with no pulses means no sensor (the pin's pull-down, or the
  // divider's bottom resistors, hold it there) - not maximum dust.
  if (edges == 0 && digitalRead(PIN_DUST) == LOW) {
    r.dustPresent = false;
    r.dustValid = false;
    r.dust = LV_ABSENT;
    Serial.println("dust: PPD42 not detected (pin held LOW, no pulses)
- check wiring");
    return;
  }
  r.dustPresent = true;
```

```
  float ratio = lowUs / (windowMs * 10.0f);  // percent of window spent
LOW
  // Shinyei's published curve for the PPD42 (particles > 1 um)
  r.dustRatio = ratio;
  r.dustConc = 1.1f * powf(ratio, 3) - 3.8f * powf(ratio, 2) + 520.0f *
ratio + 0.62f;
  r.dustValid = millis() >= DUST_WARMUP_MS;
  Level prev = r.dust;
  r.dust = r.dustValid ? gradeDust(r.dustConc) : LV_WARMUP;
  if (r.dust == LV_HIGH && prev != LV_HIGH) beep(2);
  Serial.printf("dust: low %.3f %%  ->  %.0f pcs/0.01cf (%s)\n",
                r.dustRatio, r.dustConc, LEVEL_NAME[r.dust]);
}
```

Listing A3.3: Temperature-compensated ultrasonic ranging (C++)

```
float soundSpeedCmPerUs() {
  float t = isnan(r.tempC) ? 20.0f : r.tempC;
  return (331.3f + 0.606f * t) / 10000.0f;
}

float readSonarCm(uint8_t trig, uint8_t echo) {
  digitalWrite(trig, LOW);
  delayMicroseconds(2);
  digitalWrite(trig, HIGH);
  delayMicroseconds(10);
  digitalWrite(trig, LOW);
  uint32_t us = pulseIn(echo, HIGH, 25000);  // 25 ms ~= 4.3 m, the
sensor's limit
  if (us == 0) return NAN;
  return us * soundSpeedCmPerUs() / 2.0f;
}
```

Listing A3.4: Closed-form leg inverse kinematics (Python)

```
def leg_ik(px, py, pz, side):
    """Joint angles (abduction, hip, knee) that put the foot at (px,
py, pz)."""
    d2 = py * py + pz * pz
    h = math.sqrt(max(d2 - L0 * L0, 1e-9))           # leg length
projected on the y-z plane
    abd = wrap(math.atan2(pz, py) - math.atan2(-h, side * L0))
    r2 = px * px + h * h
    c = max(-1.0, min(1.0, (r2 - L1 * L1 - L2 * L2) / (2 * L1 * L2)))
    knee = -math.acos(c)
    hip = math.atan2(-px, h) - math.atan2(L2 * math.sin(knee), L1 + L2
* math.cos(knee))
    return abd, hip, knee
```

Listing A3.5: Vegetation gate (Python)

```
# The model has no "not a leaf" class, so it answers confidently even
for a
# photo of a wall. This vegetation check runs first: it measures how
much of
# the frame is leaf-coloured (green through yellow, reasonably
saturated) -
# the same idea as the excess-green indices used in crop imaging.
Walls, skin,
# wood and floors fall outside that hue band.
MIN_LEAF_COVERAGE = float(os.environ.get("AGRI_MIN_LEAF", "0.12"))



def leaf_coverage(image):
```

```
    hsv = np.asarray(image.convert("RGB").resize((160,
120)).convert("HSV"), dtype=np.float32)
    hue = hsv[..., 0] * (360 / 255)
    sat, val = hsv[..., 1] / 255, hsv[..., 2] / 255
    leafy = (hue >= 38) & (hue <= 165) & (sat >= 0.20) & (val >= 0.12)
    return float(leafy.mean())
```

Listing A3.6: Centre-of-mass estimate used to place footholds (Python)

```
    def com_offset(self):
        """Whole-robot centre of mass in the torso frame (x, y),
smoothed."""
        torso = self.m.body("torso").id
        rel = self.d.subtree_com[torso] - self.d.xpos[torso]
        local = self.d.xmat[torso].reshape(3, 3).T @ rel
        prev = getattr(self, "_com", local[:2])
        self._com = 0.9 * prev + 0.1 * local[:2]
        return float(self._com[0]), float(self._com[1])
```

Appendix 4
Operating Instructions

All software is in the public repository github.com/AdithyaSM31/Precision-Agriculture-Monitoring-Quadruped-Robot-Dog. The steps below start the complete system on a Windows laptop.

1. Turn on the Windows Mobile Hotspot and set its band to 2.4 GHz; the ESP32 boards cannot see 5 GHz networks.
2. Copy `secrets.example.h` to `secrets.h` in each firmware folder and enter the hotspot name and password. These files are excluded from version control.
3. Flash `firmware/sensor_node` to the XIAO (board XIAO_ESP32S3) and `firmware/camera_node` to the ESP32-CAM (board AI Thinker ESP32-CAM). The XIAO's external antenna must be attached.
4. Run `dashboard/setup.bat` once, then `dashboard/run.bat` to start the dashboard; it opens http://localhost:5000. Allow Python through the Windows firewall when prompted. Run it once with internet access so the classifier can download.
5. Power both boards. The sensor node shows its address on the screen and appears on the dashboard within about ten seconds; the camera registers itself within about fifteen.
6. To scan a leaf, hold it 10–15 cm from the camera against a plain background and press Scan leaf now. Images can also be uploaded.
7. To run the simulation, double-click `simulation/run_sim.bat`; on first run it installs MuJoCo. `run_sim.bat --dashboard http://localhost:5000` also sends the simulated readings to the dashboard, and `python record.py --sweep` regenerates the video, plots and speed sweep.

Appendix 5
Bill of Materials

Table A5.1 lists the components of the final system. Prices are those paid or listed at the time of purchase; components already available to the team are marked as such.

Table A5.1: Bill of materials

Item	Qty	Cost	Status
3D-printed chassis (PLA+, TPU), 35 parts	1 set	₹5,200	Printed
Seeed XIAO ESP32-S3 with expansion board and antenna	1	Available to the team	In use
Sensirion SGP30 gas sensor	1	Available to the team	In use
Grove MCP9808 temperature sensor	1	Available to the team	In use
Shinyei PPD42 dust sensor	1	Available to the team	Wiring pending
HC-SR04 ultrasonic sensor	2	Available to the team	Wiring pending
ESP32-CAM (OV3660)	1	₹689	In use
ESP32-CAM-MB programming adapter	1	₹119	In use
Grove Relay v1.3	1	Available to the team	Planned
Resistors 1 kΩ, breadboard, jumper wires, Grove cables	—	Available to the team	In use
MG996R servo	12	₹333 each (listed)	For the walking MVP
PCA9685 16-channel servo driver	1	₹249 (listed)	For the walking MVP